



00. Ingeniería de Software

🕒 Date Created @20 de septiembre de 2025 11:56

Software es un set de programas y la documentación que acompaña. Existen tres tipos:

- System software
- Utilitarios
- Software de aplicación



Multi-person construction of multi-version software

El software es menos predecible

- No hay producción en masa, casi ningún producto de software es igual a otro.
- No todas las fallas son errores
- El software no se gasta
- El software no está gobernado por las leyes de la física

Problemas con el software

- La versión final del producto no satisface las necesidades del cliente
- No es fácil extenderlo y/o adaptarlo. Agregar más funcional en otra versión es casi una misión imposible
- Mala documentación
- Mala calidad
- Más tiempos y costos que los presupuestados

Riesgos

- Requerimientos incompletos
- Falta de involucramiento del usuario
- Falta de recursos
- Expectativas poco realistas
- Falta de apoyo de la gerencia
- Requerimientos cambiantes

Oportunidades

- Involucramiento del usuario
- Apoyo de la gerencia
- Enunciado claro de los requerimientos
- Planeamiento adecuado
- Expectativas realistas
- Hitos intermedios
- Personas involucradas competentes

2. Estado actual y antecedentes: la crisis del software

- Década del 60: surgieron proyectos de software cada vez más grandes y complejos.
- Problemas principales: **plazos incumplidos, costos elevados, sistemas defectuosos.**
- Esto llevó a lo que se llamó "Crisis del Software".

- Motivó la necesidad de metodologías más rigurosas → nacimiento de la Ingeniería de Software.

SEBOK

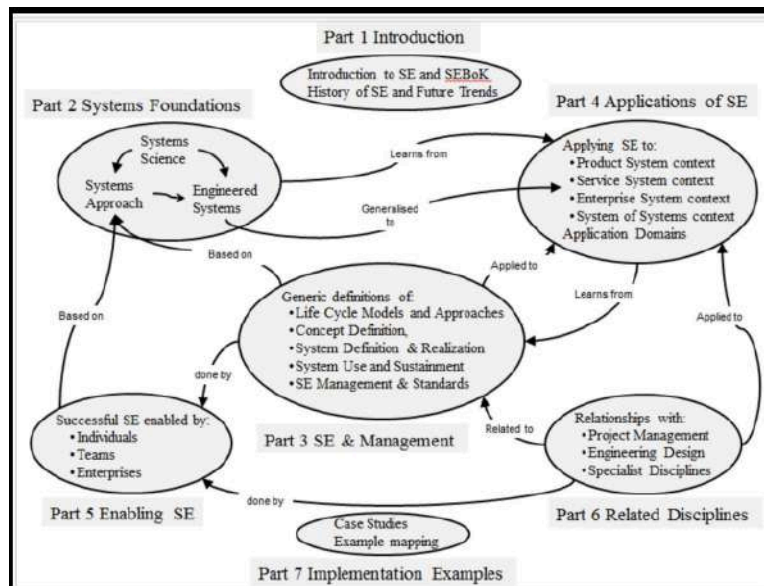
SWEBOK, Software Engineering Body of Knowledge, es un documento creado por la Software Engineering Coordinating Committee, promovido por el IEEE Computer Society, que se define como una guía al conocimiento presente en el área de la Ingeniería del Software

Los principales objetivos del **proyecto** SWEBOK incluyeron:

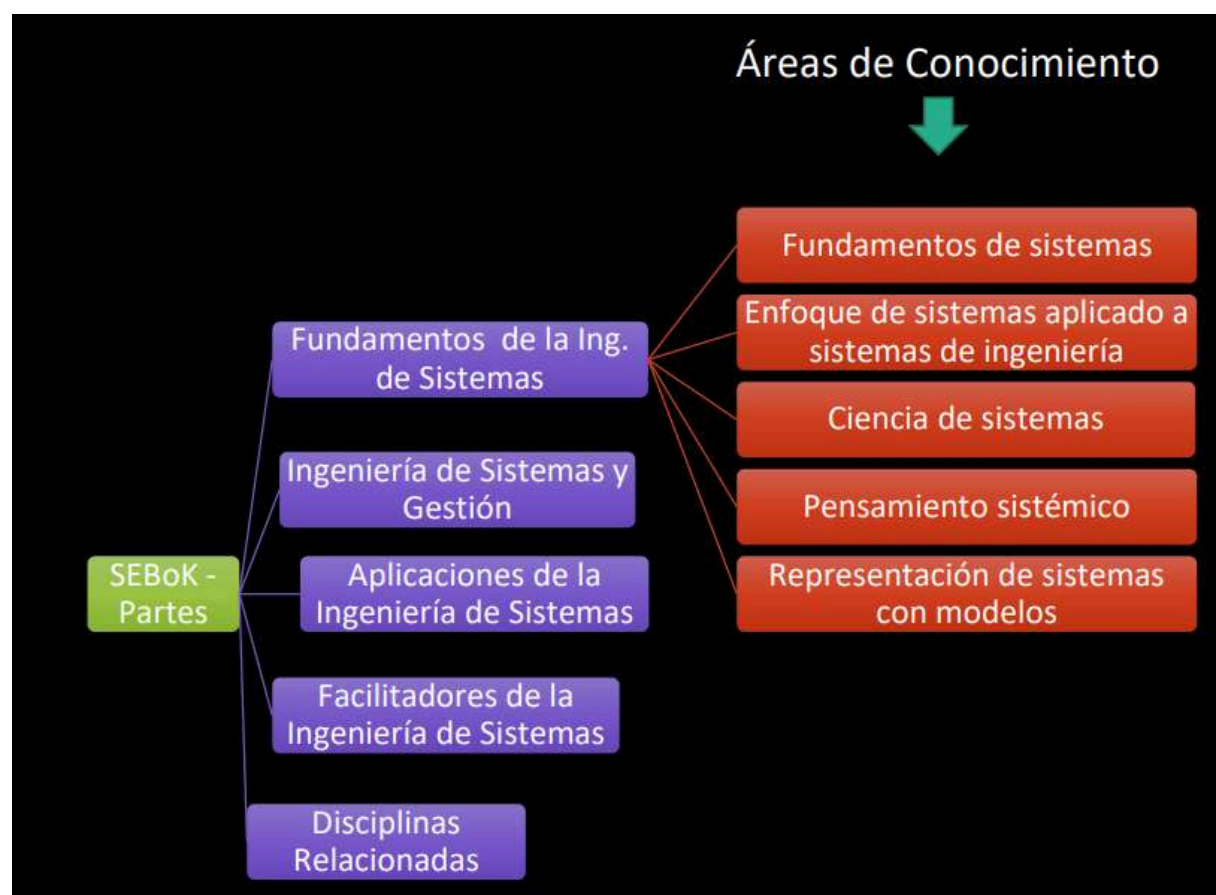
- Promover una visión consistente de la ingeniería de software en todo el mundo;
- Especificar el alcance y aclarar el lugar de la ingeniería de software en relación con otras disciplinas
- Caracterizar los contenidos de la disciplina de ingeniería de software;
- Proporcionar acceso temático al cuerpo de conocimientos de ingeniería de software;
- Proporcionar una base para el desarrollo curricular y el material de certificación y licencia individual

Software Requirements
Software Design
Software Construction
Software Testing
Software Maintenance
Software Configuration Management
Software Engineering Management
Software Engineering Process
Software Engineering Models and Methods
Software Quality
Software Engineering Professional Practice
Software Engineering Economics
Computing Foundations
Mathematical Foundations
Engineering Foundations

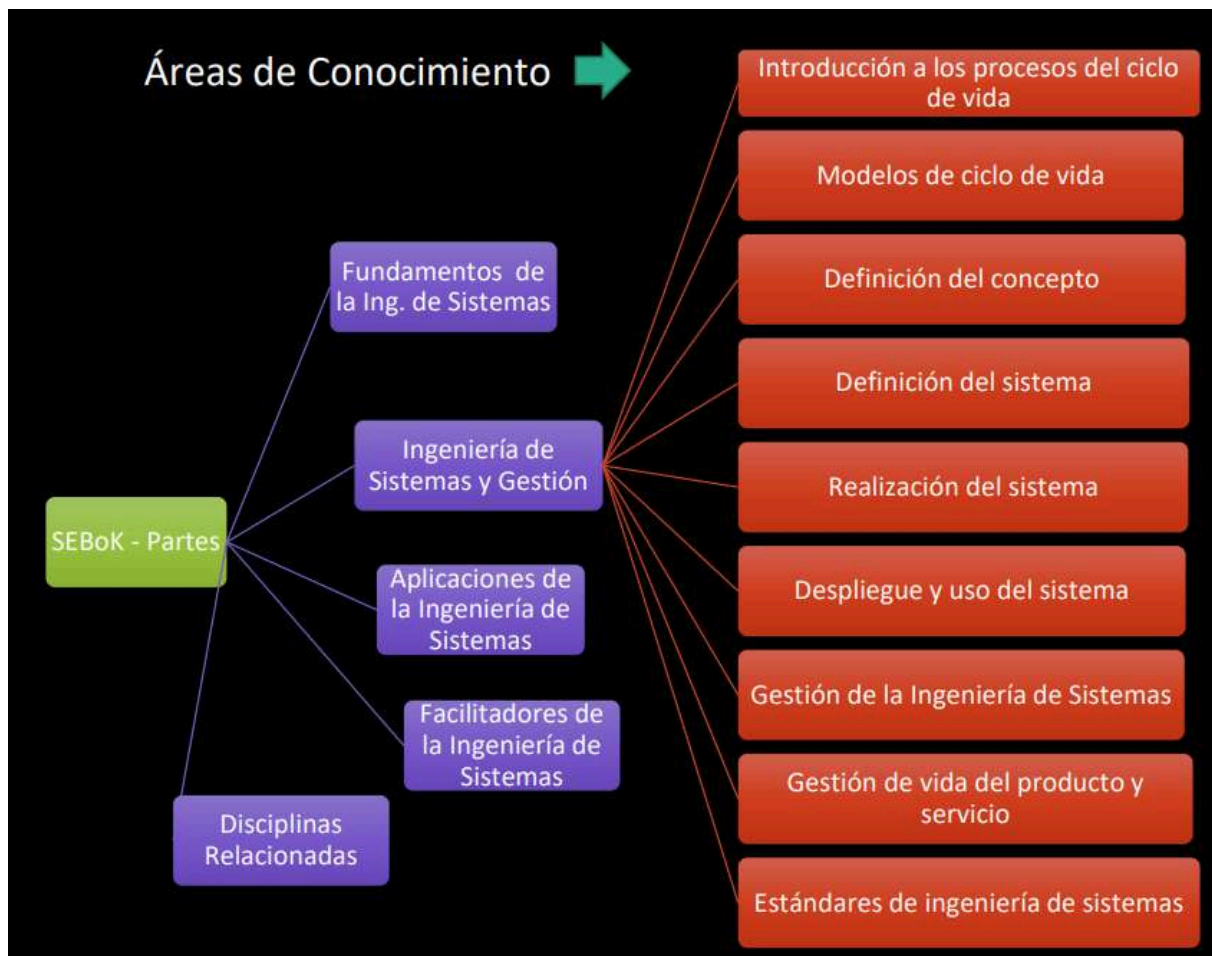
Conformado por 15 áreas de conocimiento



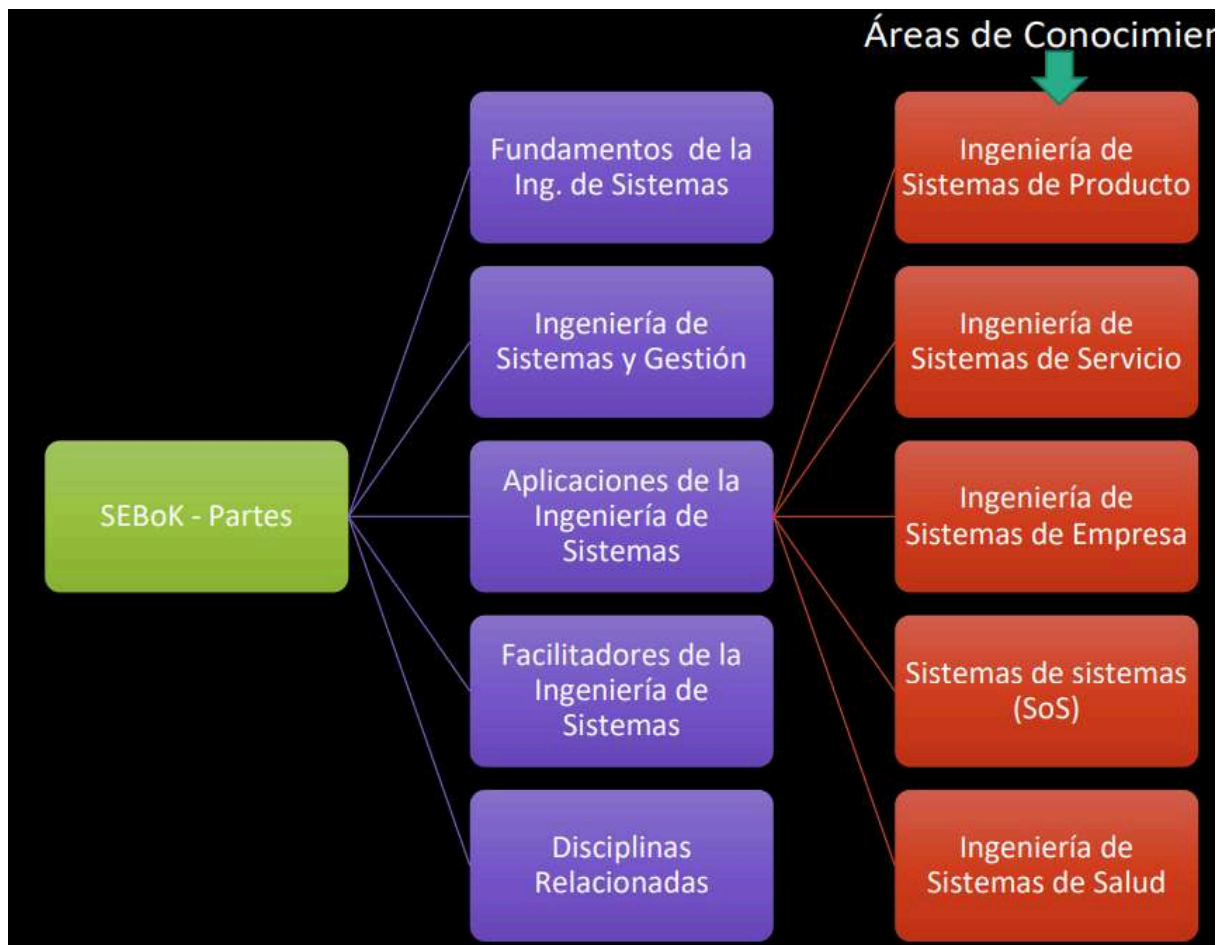
Parte 2: System Foundations



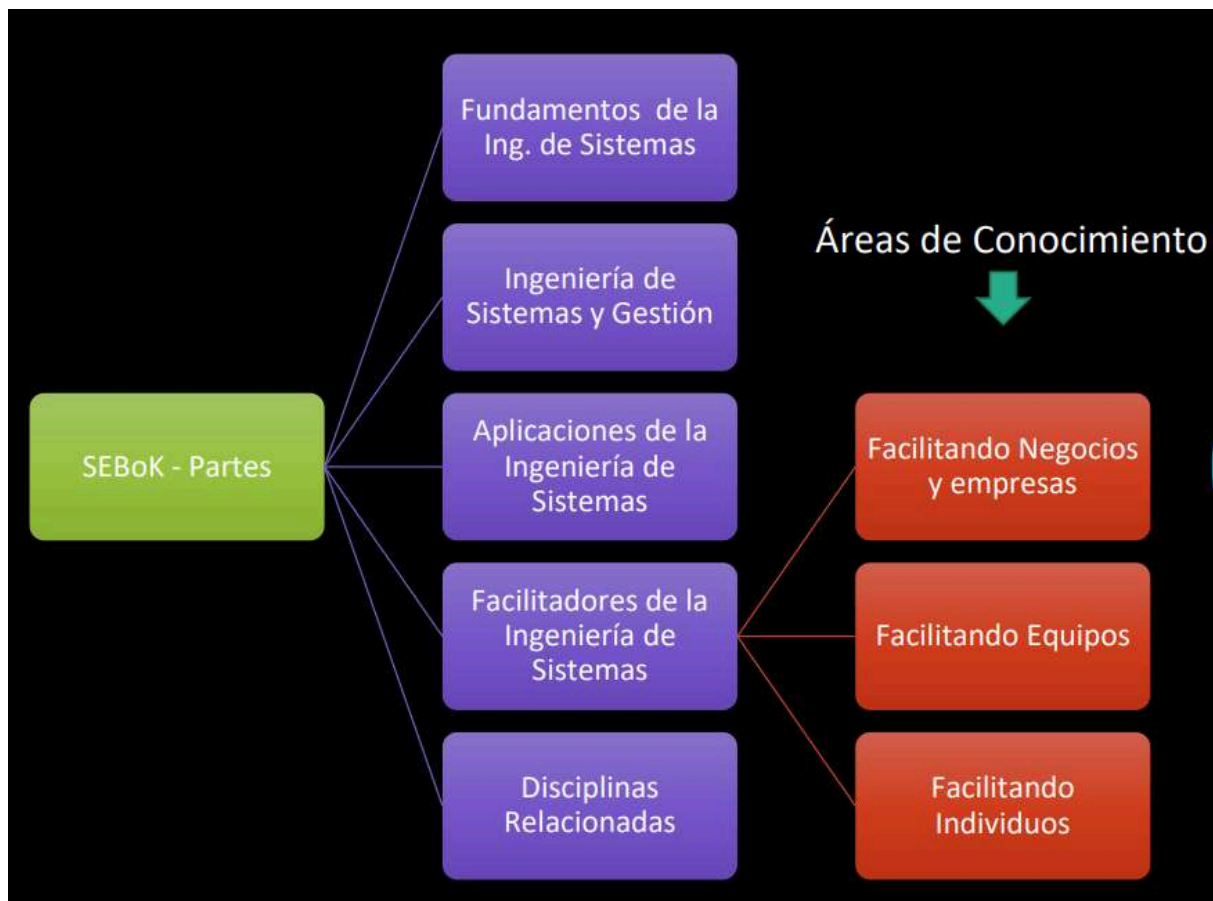
Parte 2: SE and Management



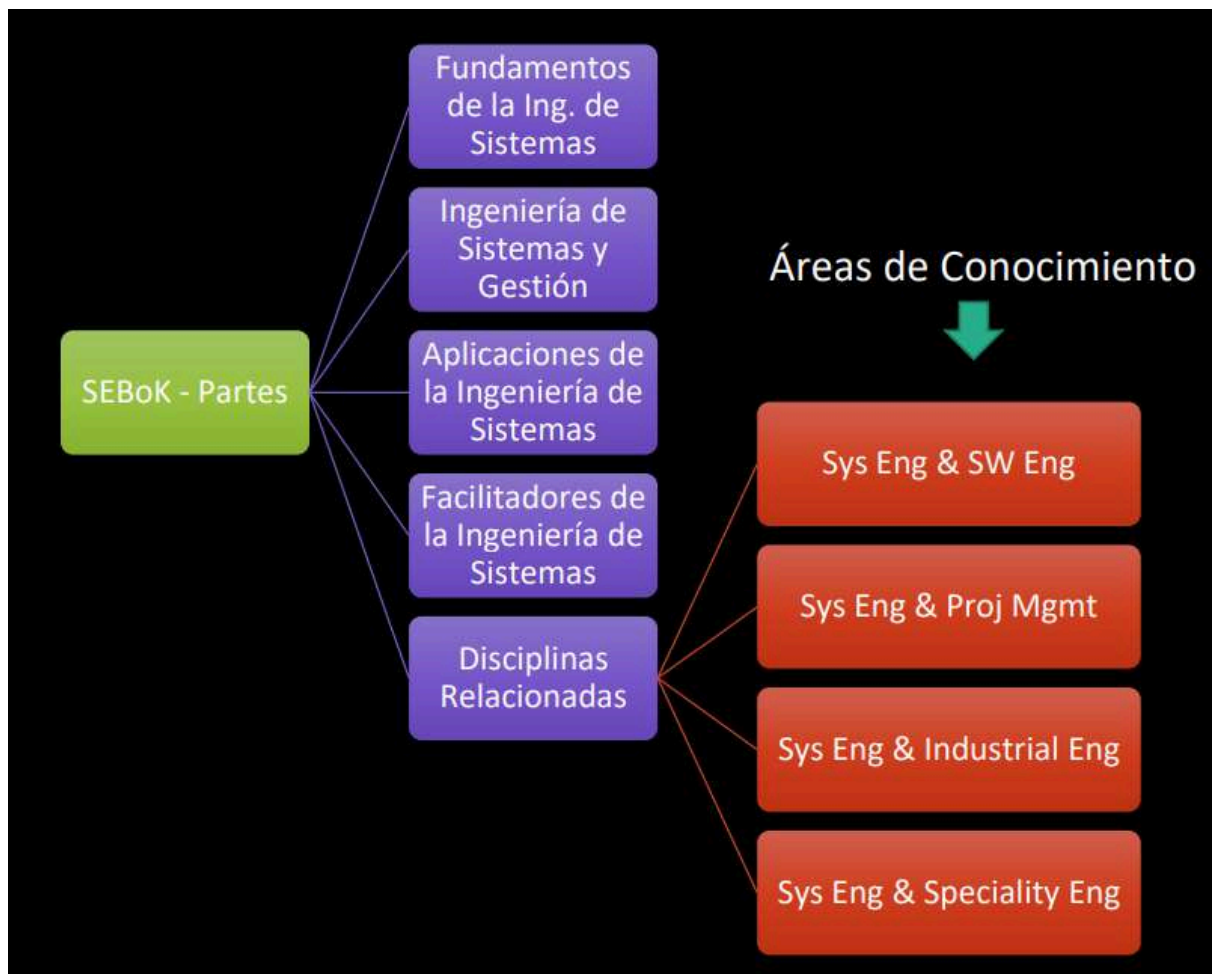
Parte 4: Applications of SE



Parte 4: Enabling SE

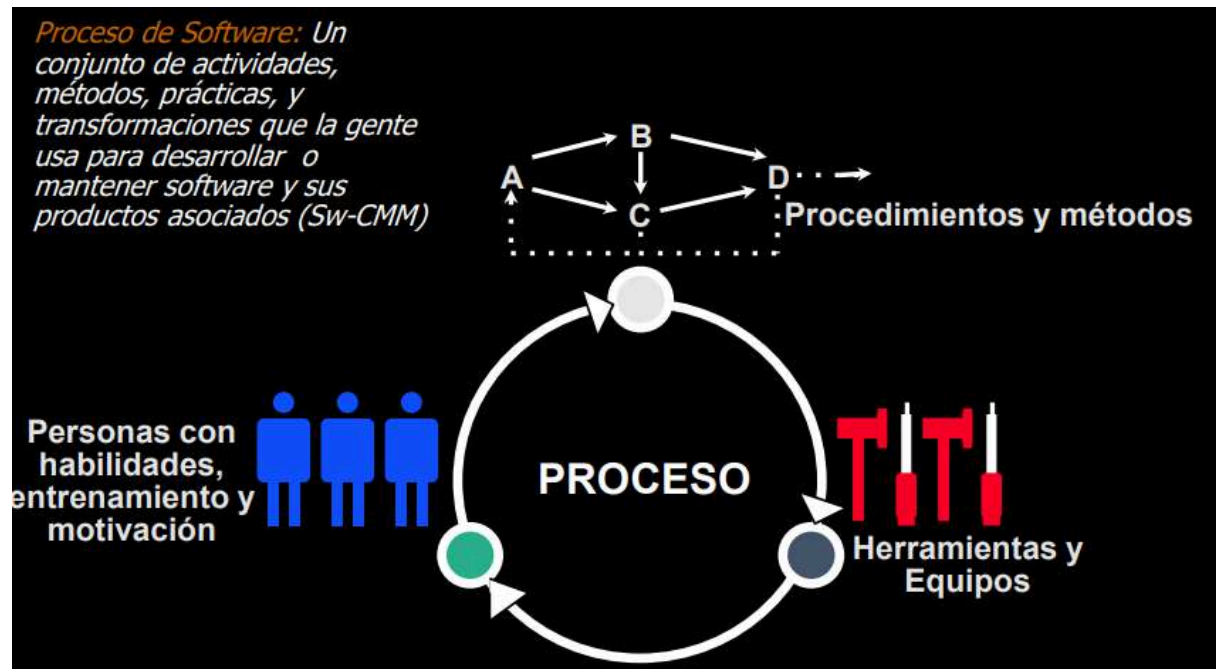


Parte 6: Related disciplines



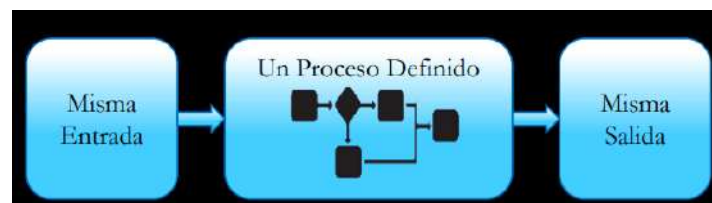
El proceso de Software

Es el conjunto estructurado de actividades para desarrollar un sistema de software. Las actividades varían dependiendo de la organización del equipo y del tipo de sistema que debe desarrollarse.



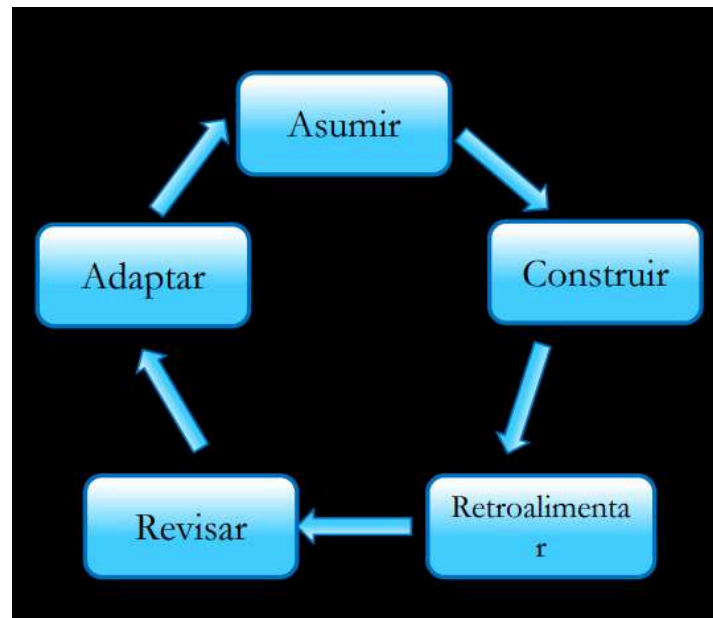
Procesos definidos

Asume que podemos repetir el mismo proceso una y otra vez, indefinidamente, y obtener los mismos resultados. La administración y control provienen de la predictibilidad del proceso definido.



Procesos empírico

Asume procesos complicados con variables cambiantes. Cuando se repite el proceso, se pueden llegar a obtener resultados diferentes. La administración y control es a través de inspecciones frecuentes y adaptaciones



Ciclos de vida

Un ciclo de vida de un proyecto software es una representación de un proceso. Grafica una descripción del proceso desde una perspectiva particular

Hay tres tipos básicos de ciclos de vida:

- **Secuencial**
 - Este modelo dispone de las actividades de forma lineal, es decir, que el proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada.
 - Se suele utilizar en aquellos sistemas donde los requerimientos se comprenden bien y son exhaustivos ya que éstos se congelan al finalizar la etapa de toma de requerimientos.
 - El trabajo principal del producto es la documentación del software entre las distintas fases.
- **Iterativo/Incremental**
 - Este modelo aplica sucesivas iteraciones en forma escalonada a medida que avanza el calendario de actividades.

- Cada iteración produce un incremento de software funcional potencialmente entregable.
- Usualmente los primeros incrementos incluyen las funciones básicas/críticas que más requiere el cliente.
- Este enfoque vincula las actividades de especificación, desarrollo y validación.
- El sistema se desarrolla como una serie de versiones (incrementos) y cada una añade funcionalidades a la versión anterior.
- Este ciclo de vida se utiliza en metodologías ágiles
- **Recurso**
 - Es utilizado para gestionar los riesgos del desarrollo de sistemas complejos a gran escala.
 - Requiere de la intervención del cliente.
 - El modelo en espiral es un modelo recursivo, que consta de 4 etapas recursivas:
 - Determinar objetivos.
 - Identificar y resolver los riesgos.
 - Desarrollar y probar.
 - Planificar la próxima iteración.
 - Para llevarlo adelante, se subdivide el proyecto en varios mini proyectos, intentando resolver en cada uno los riesgos más relevantes hasta que no quede ninguno.
 - Se generan productos independientes de la implementación, que pueden ser reusables en sistemas de características similares.

5. Ciclos de vida y su influencia en la Administración de Proyectos de Software

El **ciclo de vida** elegido determina cómo se planifican, organizan y controlan las actividades de un proyecto. Su impacto en la administración es clave porque define:

- **Planificación:** establece cuándo y cómo se llevan a cabo las fases del proyecto.

- **Gestión de riesgos:** algunos modelos (ej. en espiral) los gestionan explícitamente, otros los dejan implícitos (ej. cascada).
- **Flexibilidad:** los modelos iterativos permiten adaptarse a cambios, mientras que los secuenciales requieren mayor estabilidad de requisitos.
- **Control y seguimiento:** influyen en la cantidad de entregables, revisiones y métricas.
- **Participación del usuario:** en cascada puede ser mínima; en iterativos y ágiles, es constante.

👉 En resumen:

- **Modelos secuenciales** → favorecen la previsibilidad, pero complican la adaptación.
- **Modelos iterativos/incrementales** → facilitan el ajuste continuo, pero requieren mayor involucramiento y disciplina del equipo.
- **Modelos recursivos (espiral)** → equilibran control y flexibilidad, pero son más costosos de administrar.

7. Ventajas y desventajas de cada ciclo de vida

Modelo Secuencial (Cascada)

Ventajas

- Estructura simple y fácil de entender.
- Claridad en fases y entregables.
- Adecuado cuando los requisitos están bien definidos desde el inicio.
- Facilita el control administrativo y contractual.

Desventajas

- Muy rígido ante cambios.
- Los errores se detectan tarde (al final).
- No involucra al cliente durante la mayor parte del proceso.
- Documentación excesiva y pesada.

Modelo Iterativo / Incremental

Ventajas

- Entregas parciales frecuentes, con retroalimentación temprana.
- Reduce riesgos porque los problemas se detectan en iteraciones cortas.
- Involucra activamente al cliente.
- Funcionalidad crítica disponible en primeras versiones.

Desventajas

- Exige alta comunicación y coordinación con el cliente.
 - Requiere un equipo con mayor disciplina y experiencia.
 - Riesgo de que la integración de incrementos sea problemática.
 - Puede generar desgaste si no se planifican bien los ciclos.
-

Modelo Recursivo (Espiral)

Ventajas

- Ideal para proyectos grandes y complejos.
- Gestión de riesgos integrada en el proceso.
- Flexibilidad para cambios y refinamientos.
- Permite reutilizar componentes en proyectos similares.

Desventajas

- Costoso y de difícil implementación.
 - Complejo de administrar (necesita alta madurez en procesos).
 - No recomendable para proyectos pequeños o con bajo presupuesto.
-

8. Criterios para elección de ciclos de vida

La elección del modelo de ciclo de vida depende de las **características del producto, el proyecto y el contexto organizacional**. Algunos criterios clave son:

- **Definición de requisitos**
 - Bien definidos y estables → modelo secuencial.
 - Poco claros, cambiantes → modelo iterativo o espiral.

- **Tamaño y complejidad del proyecto**
 - Proyectos pequeños → modelos simples (secuencial o incremental).
 - Proyectos grandes/criticidad alta → espiral o enfoques iterativos con fuerte gestión de riesgos.
- **Riesgos**
 - Bajo riesgo → cascada puede ser suficiente.
 - Riesgo alto/tecnología novedosa → espiral para gestionar riesgos en cada iteración.
- **Participación del cliente**
 - Baja disponibilidad → cascada (aunque con riesgo de desalineación).
 - Alta disponibilidad → incremental o ágil.
- **Plazos y entregas**
 - Necesidad de entregas tempranas → incremental.
 - Un solo entregable final → secuencial.
- **Presupuesto y recursos**
 - Limitados → modelos simples.
 - Amplios → modelos más complejos y flexibles.

👉 La clave está en equilibrar **estabilidad de requisitos, gestión de riesgos, recursos disponibles y nivel de interacción con el cliente.**

- 1 **Apple** (881.980 millones de dólares)
- 2 **Microsoft** (803.090 millones de dólares)
- 3 **Amazon** (739.460 millones de dólares)
- 4 **Alphabet** (711.940 millones de dólares)
- 5 **Berkshire Hathaway** (536.530 millones de dólares)
- 6 **Johnson & Johnson** (396.210 millones de dólares)
- 7 **Alibaba** (387.610 millones de dólares)
- 8 **Facebook** (378.050 millones de dólares)
- 9 **JPMorgan Chase & Co** (368.560 millones de dólares)
- 10 **Tencent Holdings** (353.670 millones de dólares)

Las 10 empresas más grandes del mundo en 2019

<https://www.ig.com/es/estrategias-de-trading/las-10-empresas-mas-grandes-del-mundo-por-capitalizacion-bursati-190124>

6

Errores informáticos más costosos de la historia

<https://www.javiergarzas.com/2013/05/top-7-de-errores-informaticos.html>

1 – La destrucción del Mariner I (1962). 18,5 millones de dólares.

El del Mariner I, una sonda espacial que se dirigía a Venus, se desvió de la trayectoria de vuelo prevista poco después del lanzamiento. Desde control se destruyó la sonda a los 293 segundos del despegue. La causa fue una fórmula manuscrita que se programó incorrectamente.

2 – La catástrofe del Hartford Coliseum (1978). 70 millones de dólares.

Apenas unas horas después de que miles de aficionados abandonaron el Hartford Coliseum, el techo se derrumbó por el peso de la nieve. La causa: cálculo incorrecto introducido en el software CAD utilizado para diseñar el coliseo.

3 – El gusano de Morris (1988). 100 millones de dólares.

El estudiante de posgrado Robert Tappan Morris fue condenado por el primer ataque con "gusanos" a gran escala en Internet. Los costos de limpiar el desastre se cifran en 100 millones de dólares. Morris, es hoy profesor en MIT.

4 – Error de cálculo de Intel (1994). 475 millones de dólares.

Un profesor de matemáticas descubrió y difundió que había un fallo en el procesador Pentium de Intel. La sustitución de chips costó a Intel 475 millones.

5 – Explosión del cohete Ariane 5 (1996). 500 millones de dólares.

En el 1996, el cohete Ariane 5 de la Agencia Espacial Europea estalló. El Ariane explotó porque un número real de 64 bits (coma flotante) relacionado con la velocidad se convirtió en un entero de 16 bits. *Se dejó un poco que explica lo sucedido.*

6 – Mars Climate Orbiter (1999). 655 millones de dólares.

En 1999 los ingenieros de la NASA perdieron el contacto con la Mars Climate Orbiter en su intento que orbitase en Marte. La causa, un programa calculaba la distancia en unidades inglesas (pulgadas, pies y libras), mientras que otro utilizó unidades métricas.

7 – El error en los frenos de los Toyota (2010). 3 billones de dólares.

Toyota retiró más de 400.000 de sus vehículos híbridos en 2010, por un problema software, que provocaba un retraso en el sistema anti-bloqueo de frenos. Se estima entre sustituciones y demandas el error le costó a Toyota 3 billones de dólares.

8 – Las migraciones por el año 2000. 296,7 billones de dólares.

Se esperaba que el bug Y2K paralizase al mundo a la medianoche del 1 de enero 2000, ya que mucho software no había sido previsto para trabajar con el año 2000. El mundo no se acabó, pero se estima que se gastaron 296,7 billones de dólares para mitigar los daños.

8

01. SCM

🕒 Date Created @20 de septiembre de 2025 11:56

El software en contexto

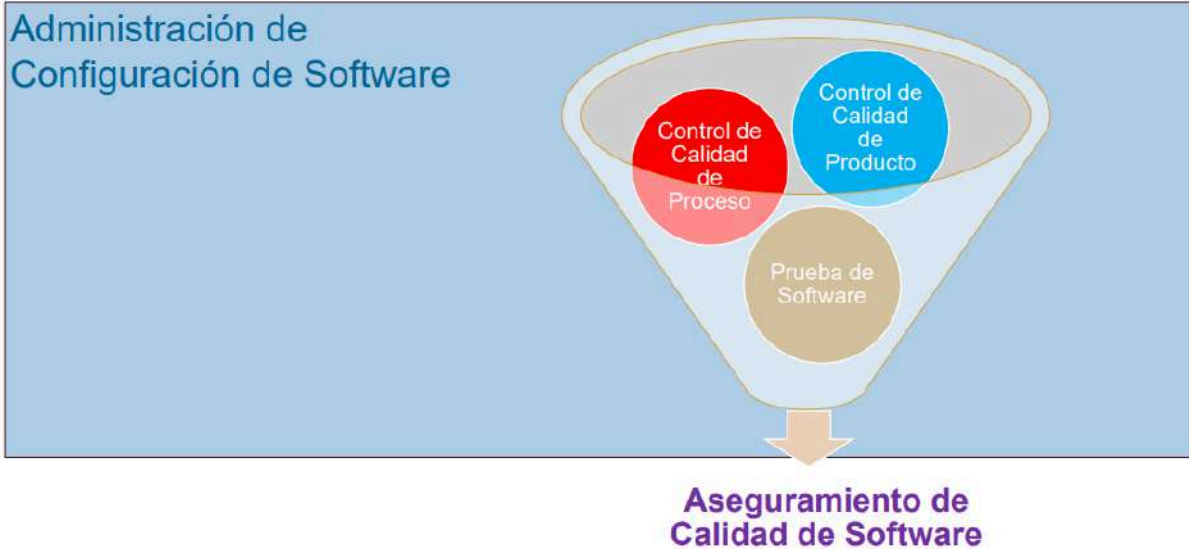


Cambios en el software

Tienen su origen en:

- Cambios del negocio y nuevos requerimientos
- Soporte de cambios de productos asociados
- Reorganización de las prioridades de la empresa por crecimiento
- Cambios en el presupuesto
- Defectos encontrados a corregir
- Oportunidades de mejora

Disciplinas de soporte



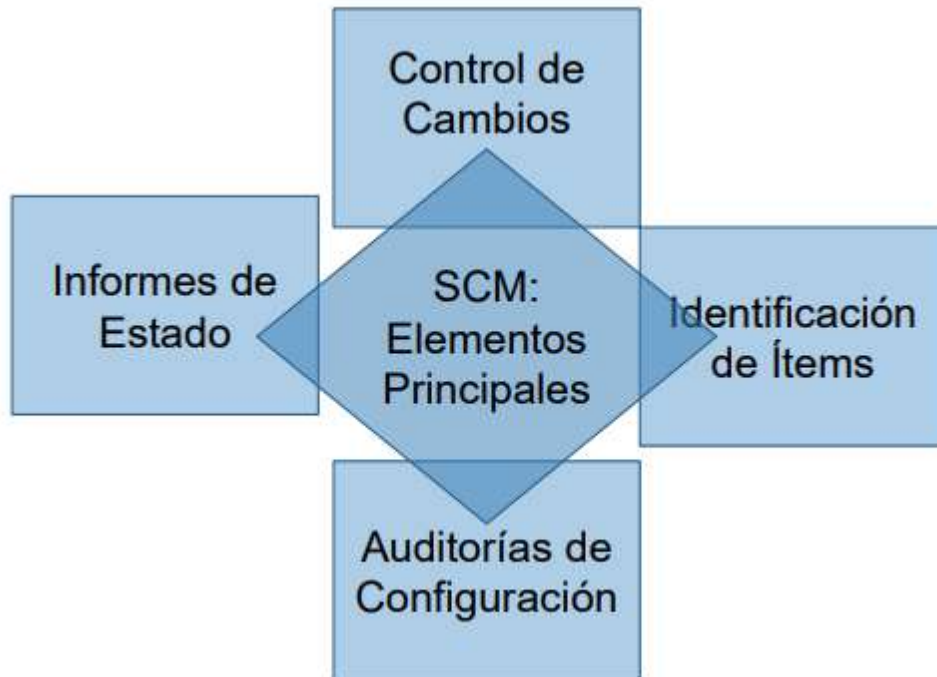
SCM como disciplina de soporte



Una disciplina que aplica **dirección y monitoreo administrativo y técnico** a:

- Identificar y documentar las características funcionales y técnicas de los ítems de configuración
- Controlar los cambios de esas características
- Registrar y reportar los cambios y su estado de implementación
- Verificar correspondencia con los requerimientos

Sus actividades fundamentales son:



El propósito de la misma es **establecer y mantener la integridad** de los productos de software a lo largo de su **ciclo de vida**.

Integridad del producto

Sucede cuando el producto:

- Satisface las necesidades del usuario
- Puede ser fácil y completamente rastreado
- Satisface criterios de performance
- Cumple con las expectativas de costo

Problemas en el manejo de componentes

Por ejemplo:

- Pérdida de un componente
- Pérdidas de cambios realizados a un componente
- Sincronía fuente-objeto-ejecutable
El código fuente no está alineado con los objetos compilados o el ejecutable final.
- Regresión de fallas

Un bug que ya había sido solucionado reaparece

- Doble mantenimiento por copias duplicadas, llevando a la inconsistencia
- Superposición de cambios
- Cambios no validados

Conceptos clave

Ítem de configuración

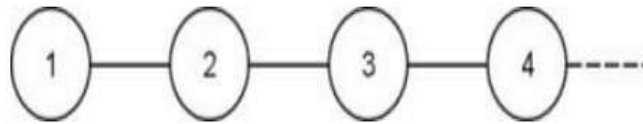
Todos los artefactos que forman parte del producto o del proyecto, que pueden sufrir cambios o necesitan ser compartidos entre los miembros del equipo y sobre los cuales se necesita conocer su estado y evolución

Por ejemplo:

- Plan de CM
- Propuestas de cambio
- Código fuente
- Casos de prueba
- Prototipo de interfaz
- Instructivo de ensamble
- Requerimientos
- Manual de usuario

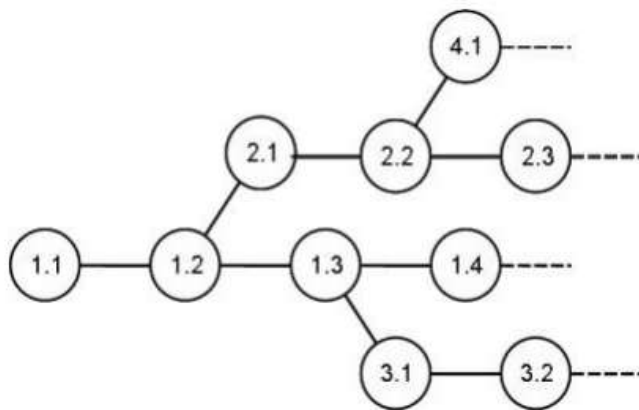


La **versión** de un ítem se define como la forma particular de un artefacto en un instante o contexto dado. Se refiere a la evolución de un único ítem



Evolución lineal de un ítem de configuración

La **variante** es una versión de un ítem que evoluciona por separado, es decir, representan configuraciones alternativas



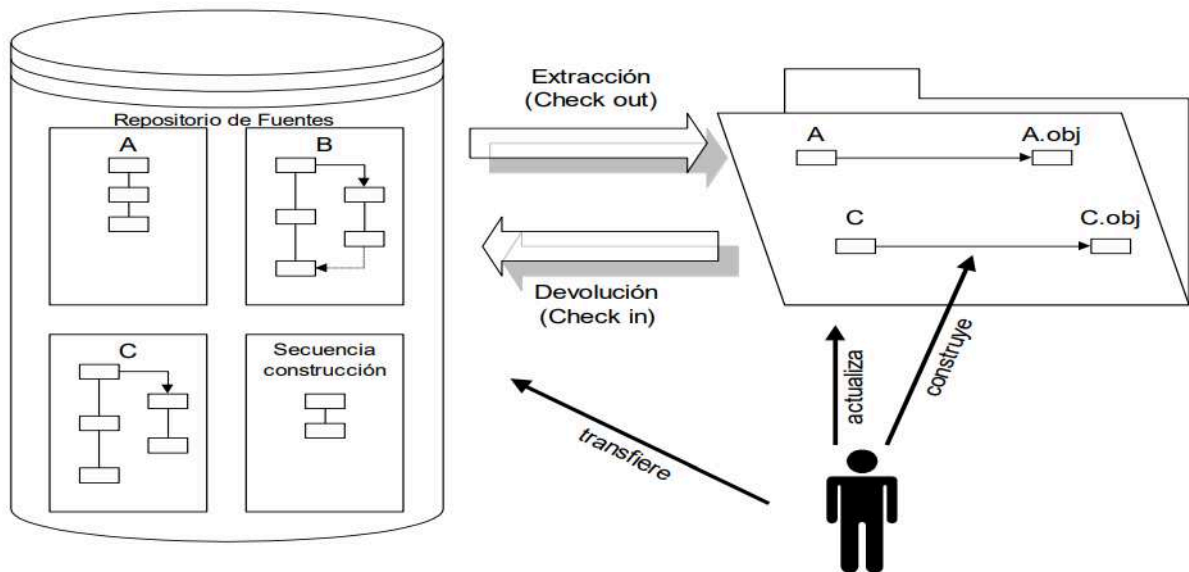
Variante de un ítem de configuración



Entonces, la configuración del software está definida por un conjunto de ítems de configuración con su correspondiente versión en un momento determinado

Repositorio

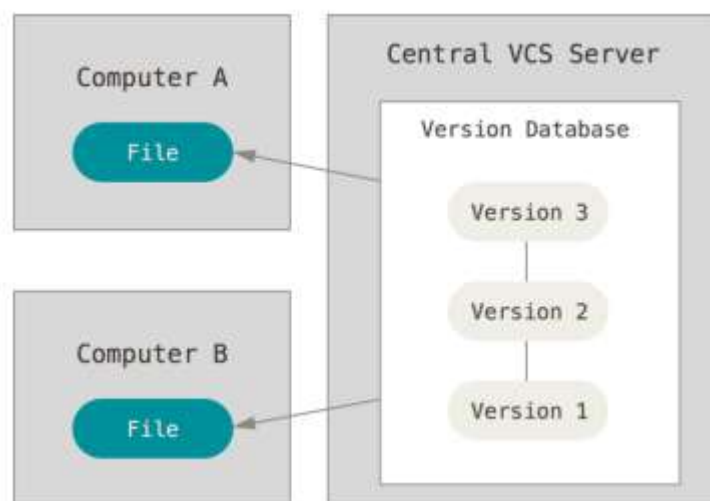
Es el contenedor de los ítems de configuración, su historia, sus atributos y relaciones. Es utilizado para hacer evaluaciones de impacto de los cambios propuestos.



Existen dos tipos:

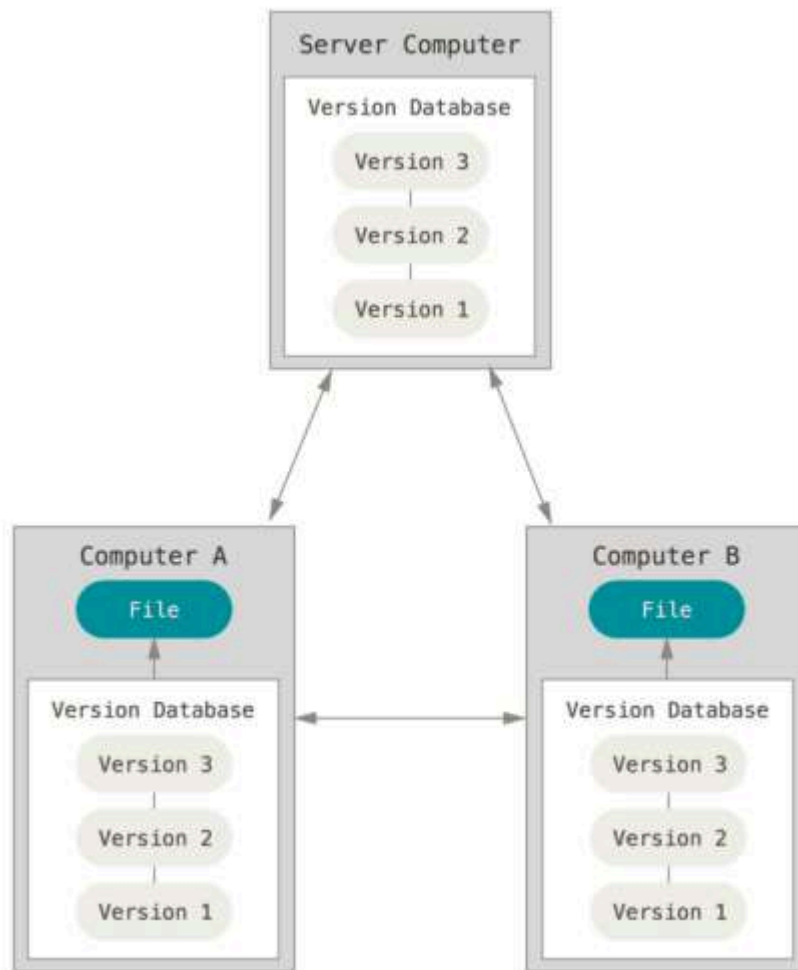
- **Centralizados**

Un servidor contiene todos los archivos con sus versiones



- **Descentralizados**

Cada cliente tiene una copia exactamente igual del repositorio completo



Línea base

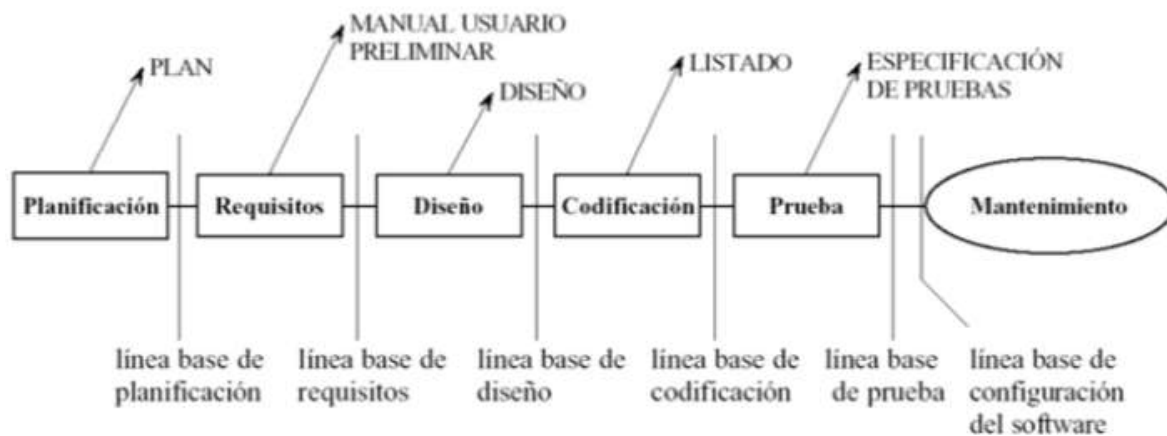
Representa una configuración que ha sido revisada formalmente y sobre la que se ha llegado a un acuerdo

Sirve como base para desarrollos posteriores y puede cambiarse sólo a través de un procedimiento formal de control de cambios

Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto

Pueden ser:

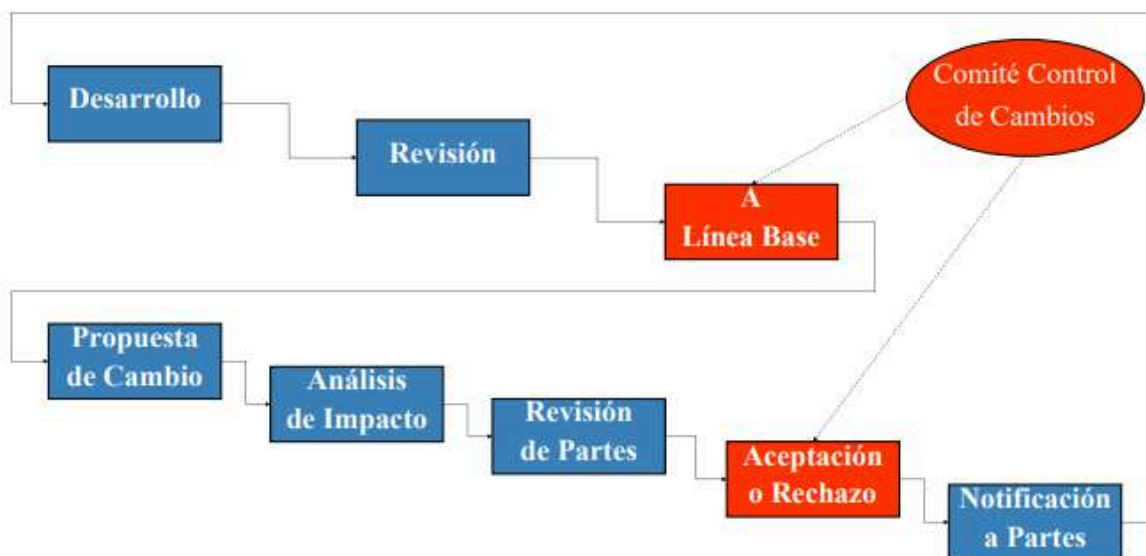
- De **especificación**
- De **productos** que han pasado por un control de calidad definido previamente



Ramas

Sirven para bifurcar el desarrollo, permitiendo la experimentación.

Control de cambios

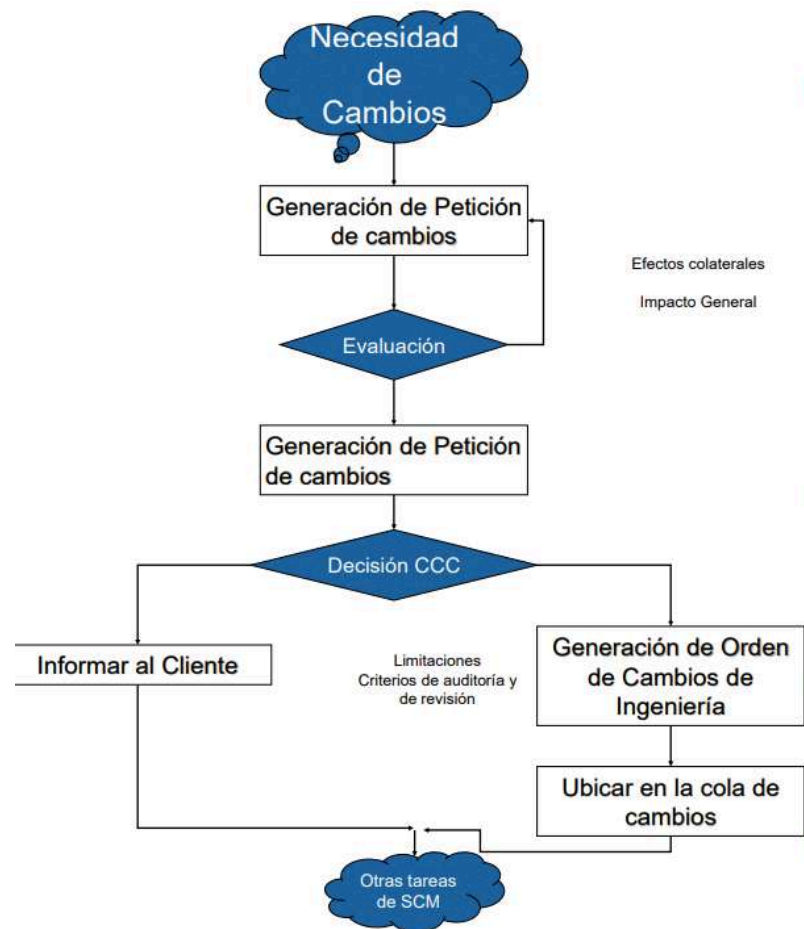


Tiene su origen en un **requerimiento de cambio** a uno o varios ítems de configuración que se encuentran en una línea base.

Es un **procedimiento formal** que involucra diferentes actores y una evaluación del impacto del cambio

El **Comité de Control de Cambios** está formado por representantes de todas las áreas involucradas en el desarrollo:

- Análisis, Diseño
- Implementación
- Testing
- Otros interesados



Auditorías de configuración de software

Existen dos tipos:

- **Auditoría física de configuración** (PCA)

Asegura que lo que está indicado para cada ICS en la línea base o en la actualización se ha alcanzado realmente

- **Auditoría funcional de configuración** (FCA)

Evaluación independiente de los productos de software, controlando que la funcionalidad y performance reales de cada ítem de configuración sean consistentes con la especificación de requerimientos.



Sirve a dos procesos básicos:

- **Validación**

El problema es resuelto de manera apropiada que el usuario obtenga el producto correcto.

- **Verificación**

Asegura que un producto cumple con los objetivos preestablecidos, definidos en la documentación de líneas base

Registro e Informe de estado

Se ocupa de mantener los registros de la evolución del sistema. Maneja mucha información y salidas por lo que se suele implementar dentro de procesos automáticos.

Incluye reportes de rastreabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida

Responde preguntas como:

- ¿Cuál es el estado del ítem?
- ¿Un requerimiento de cambio ha sido aprobado o rechazado por el CCB?
- ¿Cuál es la diferencia entre una versión y otra dada?

Plan de Gestión de Configuración

Debe incluir:

- Reglas de nombrado de los CI
- Herramientas a utilizar para SCM
- Roles e integrantes del Comité

- Procedimiento formal de cambios
- Plantillas de formularios
- Procesos de auditoría

SCM en ambientes ágiles

- Sirve a los practicantes (equipo de desarrollo) y no viceversa
- Hace seguimiento y coordina el desarrollo en lugar de controlar a los desarrolladores
- Esforzarse por ser transparente y "sin fricción", automatizando tanto como sea posible
- Responde a los cambios en lugar de tratar de evitarlos
- Coordinación y automatización frecuente y rápida
- Eliminar el desperdicio - no agregar nada más que valor

02. Requerimientos ágiles

🕒 Date Created @20 de septiembre de 2025 11:56



- 1 Nuestra mayor prioridad es **satisfacer al cliente.**
- 2 **Aceptar** que los requisitos cambien.
- 3 **Entregar** software funcional frecuentemente.
- 4 Los responsables de negocios, diseñadores y desarrolladores deben **trabajar juntos** día a día durante el proyecto.
- 5 Desarrollamos proyectos en torno a **individuos motivados.**
- 6 El método más eficiente de comunicar información es **conversaciones cara a cara.**
- 7 El **software funcionando** es la principal **medida de éxito.**
- 8 Los procesos ágiles promueven el **desarrollo sostenible.**
- 9 La **atención continua a la excelencia técnica y al buen diseño** mejor la Agilidad.
- 10 La **simplicidad** es esencial.
- 11 Las mejores arquitecturas, requisitos, y diseños emergen de **equipos auto-organizados.**
- 12 A intervalos regulares el equipo reflexiona sobre cómo ser más efectivo y de acuerdo a esto **ajustan su comportamiento.**

Ágil

Es una **ideología con un conjunto definido de principios** que guían el desarrollo del producto



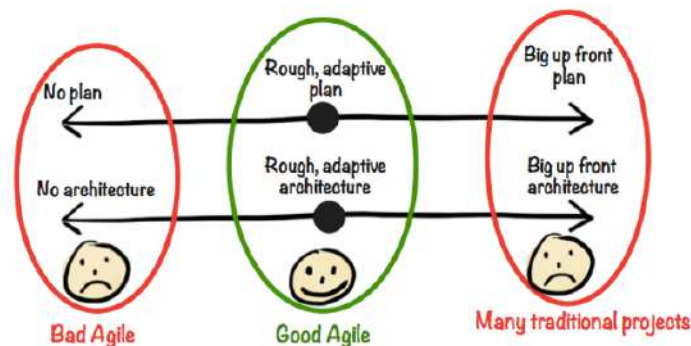
Los valores de los equipos ágiles son:

- Planificación continua, multi-nivel
- Facultados, auto-organizados, equipos completos
- Entregas frecuentes, iterativas y priorizadas
- Prácticas de ingeniería disciplinadas
- Integración continua
- Testing Concurrente

Es el balance entre ningún proceso y demasiado proceso. La diferencia inmediata es la exigencia de una menor cantidad de documentación

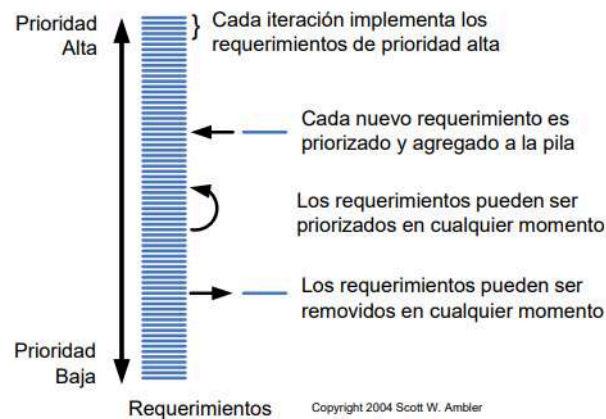
Los métodos ágiles son adaptables en lugar de predictivos y son orientados a la gente en lugar de orientados al proceso.

Don't go overboard with Agile!



Requerimientos en Agile

Los requisitos cambiantes son una ventaja competitiva si puede actuar sobre ellos



Tipos de requerimientos

- De negocio
Disminuir un X% de tiempo invertido en procesos manuales relacionados con atención al cliente.
- De usuario
Realizar consultas en línea del estado de cuenta de los clientes
- Funcional
- No funcional
- De implementación





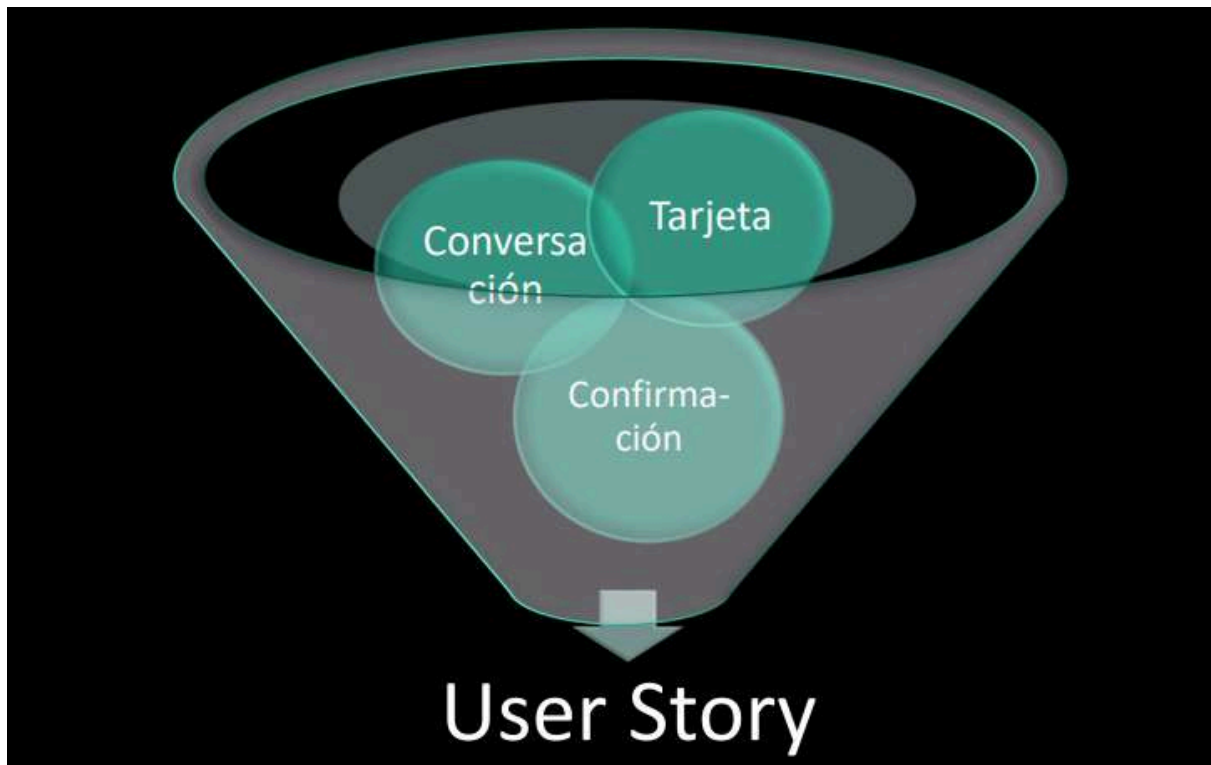
Principios ágiles relacionados a los R.A



User stories

Son:

- Una necesidad del usuario
- Una descripción del producto
- Un ítem de planificación
- Token para una conversación
- Mecanismo para diferir una conversación



Como **<nombre del rol>**, yo puedo **<actividad>**, de formal tal que **<valor de negocio que recibo>**

Modelado de roles

Puede realizarse a través de diferentes técnicas:

- Tarjeta de rol de usuario

Rol de Usuario: Reclutador Interno

Nos es un experto en computadoras, pero bastante adepto a utilizar la Web. Utilizará el software con poca frecuencia pero muy intensamente. Leerá anuncios de otras compañías para averiguar cuál es la mejor palabra para sus anuncios. La facilidad de uso es importante, pero más importante es que lo que aprenda, lo pueda recordar meses después.

- Personas

Mario trabaja como reclutador en el departamento de Speedy Networks, una fábrica de componentes de red de alta gama. El ha trabajado para Speedy Networks por 6 años. Mario tiene un arreglo de horario flexible y trabaja desde casa cada viernes. Mario es muy fuerte con las computadoras y se considera a sí mismo un usuario avanzado de los productos que usa. La esposa de Mario, Kim, está terminando su Doctorado en Química en la Universidad de Stanford. Dado que Speedy Networks ha estado creciendo consistentemente, Mario siempre está buscando ingenieros.



- Personajes extremos

Diseño de un PDA para:

- El Papa
- Una mujer de 20 años con muchos novios
- Un traficante de drogas

Tanto la mujer como el traficante desearán mantener agendas separadas en caso de que la vea la policía o un novio. El Papa probablemente tenga menos necesidad de discreción pero querrá un tamaño de fuente más grande.

- Usuarios representantes o proxies

- Gerentes de Usuarios
- Gerentes de Desarrollo
- Alguien del grupo de marketing
- Vendedores
- Expertos del Dominio
- Clientes
- Capacitadores y personal de soporte.

No son ideales como los usuarios verdaderos
...EVITELOS !!!

Criterios de aceptación

Definen los límites para una US, ayudando a que los PO respondan lo que necesitan de la US para que esta les provea valor, Ayudan a que el equipo tenga una visión compartida de la misma, y también ayudan a derivar las pruebas.

Definen una intención, no una solución. Son independientes de la implementación

Los detalles de implementación que son el resultado de las conversaciones con el PO y el equipo puede capturarlos en dos lugares:

- Documentación interna de los equipos
- Pruebas de aceptación automatizadas

Pruebas de aceptación

Expresan detalles resultantes de la conversación, complementando la US. Es un proceso de dos pasos, que involucra identificarlas y diseñar las pruebas completas que se adapten a ellos.

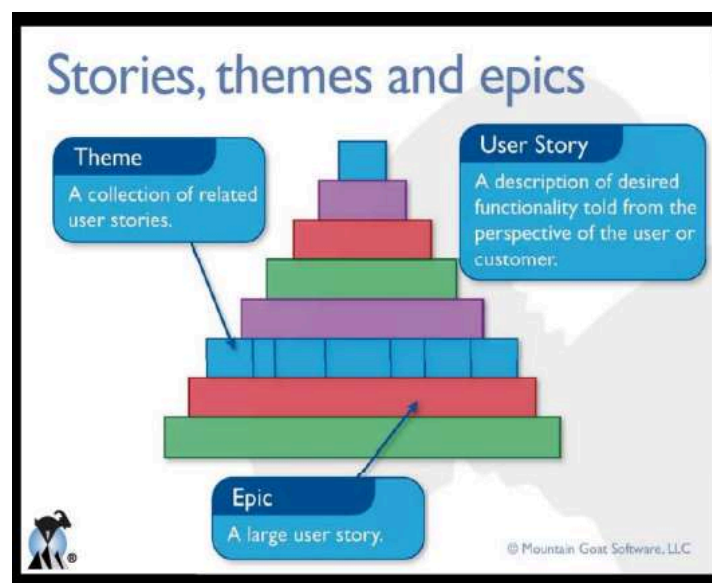
Buscar Destino por Dirección Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme. Criterios de Aceptación: • La altura de la calle es un número. • La búsqueda no puede demorar más de 30 segundos.	Pruebas de Usuario ❑ Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura existente (pasa). ❑ Probar buscar un destino en un país y ciudad existentes, de una calle inexistente (falla). ❑ Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura inexistente (falla). ❑ Probar buscar un destino en un país inexistente (falla). ❑ Probar buscar un destino en País existente, ciudad inexistente (falla). ❑ Probar buscar un destino en un país y ciudad existentes, de una calle existente y demora más de 30 segundos (falla).
--	--

INVEST Model

Son características que debe poseer una US para ser de calidad. Este modelo sirve para evaluar si la US cumple el criterio de Ready

- Independent → calendarizables e implementables en cualquier orden
- Negotiable → el qué, no el cómo
- Valuable → debe tener valor para el cliente
- Estimatable → para ayudar al cliente a armar un ranking basado en costos
- Small → deben ser consumidas en una iteración
- Testeable → se debe poder demostrar que fueron implementadas

Niveles de abstracción



Spikes

Es un tipo especial de historia, utilizado para quitar riesgo e incertidumbre de una US , y pueden utilizarse para:

- Inversión básica para familiarizar al equipo con una nueva tecnología o dominio.
- Analizar un comportamiento de una historia compleja y poder así dividirla en piezas manejables.
- Ganar confianza frente a riesgos tecnológicos, investigando o prototipando para ganar confianza.
- Frente a riesgos funcionales, donde no está claro como el sistema debe resolverla interacción con el usuario para alcanzar el beneficio esperado.

Se clasifican en:

Técnicas	Funcionales
• Utilizadas para investigar enfoques técnicos en el dominio de la solución. • Evaluar performance potencial • Decisión hacer o comprar • Evaluar la implementación de cierta tecnología. • Cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo.	• Utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema. • Usualmente son mejor evaluadas con prototipos para obtener realimentación de los usuarios o involucrados.

Deben ser:

Estimables, demostrables, y aceptables
La excepción, no la regla • Toda historia tiene incertidumbre y riesgos. • El objetivo del equipo es aprender a aceptar y resolver cierta incertidumbre en cada iteración. • Los spikes deben dejarse para incógnitas mas críticas y grandes. • Utilizar spikes como última opción.
Implementar la spike en una iteración separada de las historias resultantes • Salvo que el spike sea pequeño y sencillo y sea probable encontrar una solución rápida en cuyo caso, spike e historia pueden incluirse en la misma iteración.

03. Estimaciones ágiles

🕒 Date Created @20 de septiembre de 2025 11:57



Si las estimaciones se utilizan como compromisos son muy peligrosas y perjudiciales para cualquier organización.



Lo más beneficioso en las estimaciones es el “proceso de hacerlas”.



La estimación podría servir como una gran respuesta temprana sobre si el trabajo planificado es factible o no.



La estimación puede servir como una gran protección para el equipo.

Las features/stories son estimadas usando una medida de tamaño relativo conocida como **story points** (SP), la cual es relativa y no está basada en tiempo.

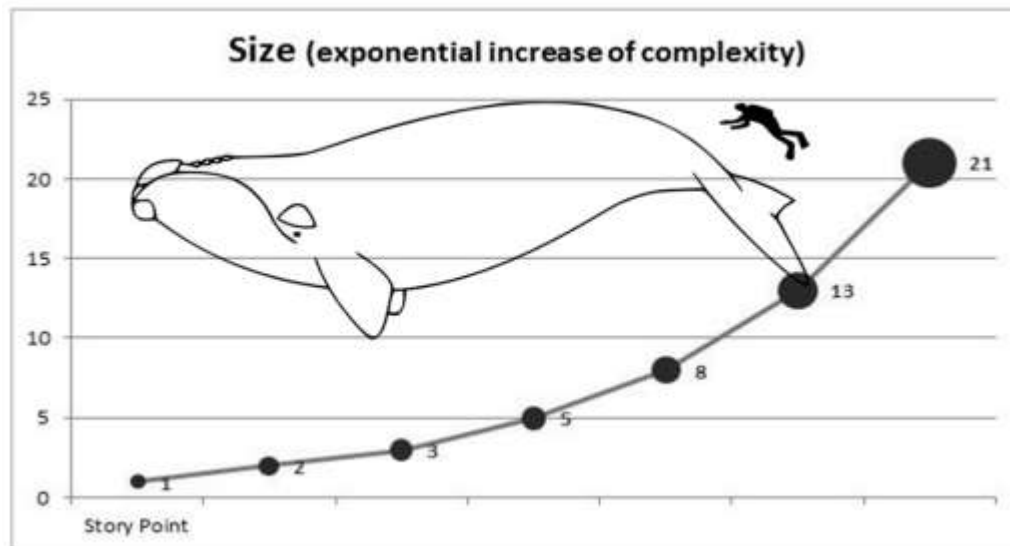
El tamaño refiere a la cantidad de trabajo necesario para producir esa feature/story o cuán compleja/grande es.

TASK DESCRIPTION vs. EFFORT

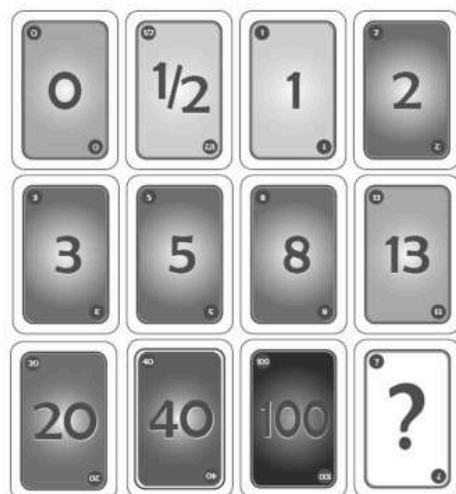


Story point

Es una unidad de medida específica (del equipo) de complejidad, riesgo y esfuerzo. Da idea del "peso" de cada story y decide cuan grande o compleja es.



¿Cómo “decodificar” las estimaciones?



- **0**: Quizás ud. no tenga idea de su producto o funcionalidad en este punto.
- **1/2, 1**: funcionalidad pequeña (usualmente cosmética).
- **2-3**: funcionalidad pequeña a mediana. Es lo que queremos. 😊
- **5**: Funcionalidad media. Es lo que queremos 😊
- **8**: Funcionalidad grande, de todas formas lo podemos hacer, pero hay que preguntarse sino se puede partir o dividir en algo más pequeño. No es lo mejor, pero todavía 😊
- **13**: Alguien puede explicar por que no lo podemos dividir?
- **20**:Cuál es la razón de negocio que justifica semejante story y más fuerte aún, por qué no se puede dividir?.
- **40**: no hay forma de hacer esto en un sprint.
- **100**: confirmación de que está algo muy mal. Mejor ni arrancar.

Poker planning

Velocity

Es una medida o métrica del progreso de un equipo. Se calcula sumando el número de story points que el equipo completa durante la iteración,

considerando únicamente aquellos que correspondan a US completadas en el mismo.



La velocidad corrige los errores de estimación

Poker estimation

Prerrequisitos:

- Lista de features/stories a ser estimadas
- Cada estimador tiene un mazo de cartas.

Pasos:

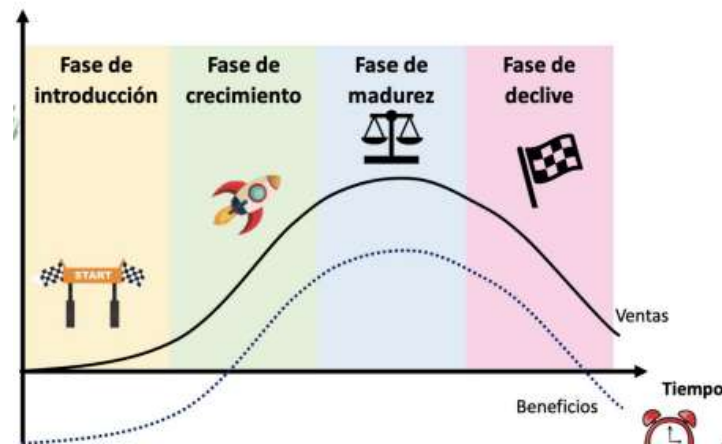
1. Determine la **base story** (la canónica) que será usada para comprar con las otras stories. Digamos, Story Z.
 - 1.1 La story a ser estimada se lee a todo el equipo.
 - 1.2 Los estimadores discuten la story, haciendo preguntas al product owner (las que se necesiten).
 - 1.3 Cada estimador selecciona una carta y pone la carta boca abajo en la mesa.
 - 1.4 Cuando todos pusieron las cartas, las mismas se exponen al mismo tiempo.
 - 1.5 Si todos los estimadores selecciona el mismo valor, ese es el estimado. Sino, los estimadores discuten sus resultados, poniendo especial atención en los más altos y los más bajos. Después de la charla, GOTO to 1.3.
2. Se toma la próxima story, se discute con el product owner.
3. Cada estimador asigna a la story un valor por comparación contra la base story. "Cuan grande/pequeña, compleja, riesgosa es esta story comparada con Story Z?". GOTO to 1.3



04. Gestión de productos

🕒 Date Created @20 de septiembre de 2025 11:57

Ciclo de vida de un producto



Evolución de los productos de software



MVP

Se define como la versión de un nuevo producto que permite a un equipo recopilar la cantidad máxima de aprendizaje validado sobre los clientes con el menor esfuerzo



Ver lo que la gente **realmente hace** con el producto es mucho más confiable que preguntar qué harían con el mismo.

Entre sus **características** se destaca que:

- Tiene el valor suficiente para que las personas estén dispuestas a usarlo o comprarlo.
- Demuestra beneficios a futuro para retener a los usuarios.
- Proporciona un ciclo de retroalimentación para guiar el desarrollo a futuro.
- Posee todas las funciones necesarias para solucionar un problema específico



Los **errores comunes** son:

- Enfatizar la parte de mínima y dejar de lado la parte de viable. Si el producto no tiene la calidad suficiente, no será posible obtener la retroalimentación del cliente.
- Confundirlo con MMF o MMP. La MMF o el MMP se enfocan en ganar, en cambio un MVP se enfoca en el aprendizaje y en satisfacer las necesidades del cliente
- Hacer del MVP un producto final

Un MVP permite validar **diferentes hipótesis**:

- **Hipótesis del valor**

Prueba si el producto realmente está entregando valor a los clientes después de que comienzan a usarlo. Un ejemplo de métrica de prueba puede ser la tasa de retención

- **Hipótesis de crecimiento**

Prueba cómo nuevos clientes descubrirán el producto. Una métrica de prueba puede ser la tasa de referencia o Net Promoter Score (NPS)

La **preparación** de un MVP requiere los siguientes pasos:

1. **Encontrar un nicho de mercado**

2. **Crear un roadmap realista**

Es una definición de *muy alto nivel* de qué características del producto se podrían tener a cierta altura del año.

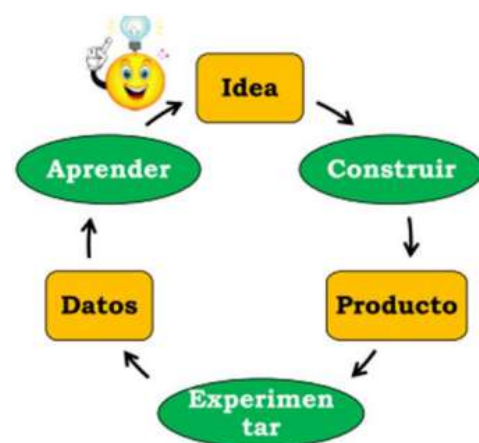
3. **Investigar la competencia**

4. **Pre-vender el MVP**

5. **Testear las suposiciones**

6. **Asegurarse de que el MVP esté resolviendo el problema correcto**

7. **Enfocarse en las funcionalidades principales**



Lean Thinking define la creación de valor como proveer beneficios a los clientes, cualquier otra cosa es desperdicio.

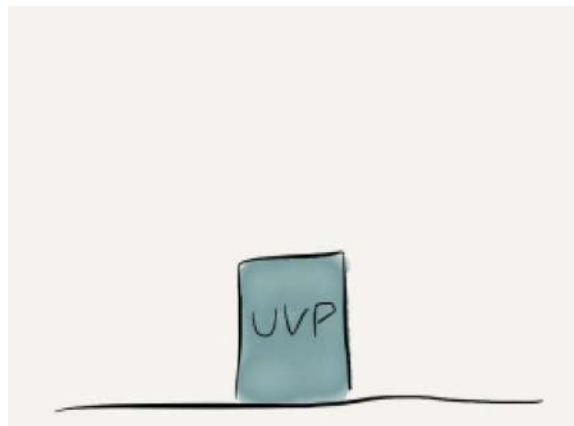
A la hora de realizar un MVP, se debe tener en cuenta la **matriz de priorización**:

	Urgente	No urgente
Importante	1. Hacer	2. Planear su desarrollo para un lanzamiento beta
No importante	3. Delegar y considerar integraciones de terceras partes	4. Eliminar del roadmap

Dinosaurio Lean/Agile

UVP → Unique Value Proposition

Comprender que un producto nuevo tiene una hipótesis de valor único, es decir, el producto será único



MVP → Minimum Viable Product

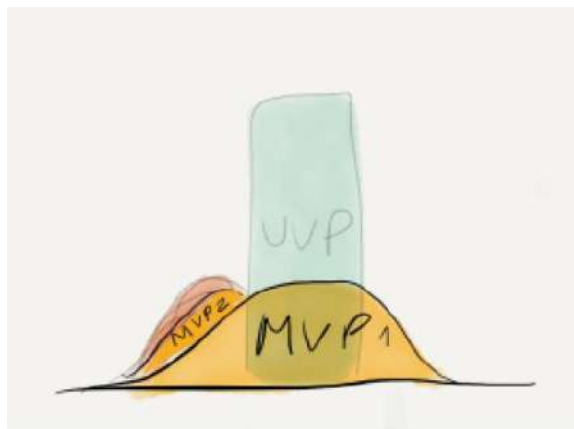
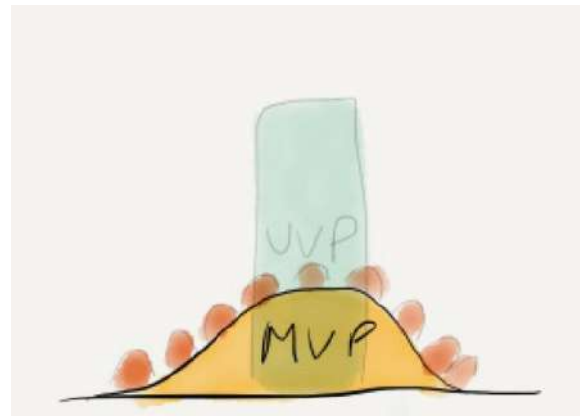


El siguiente paso es crear un MVP para probar la hipótesis.

Esto se centra en su propuesta de valor única, pero normalmente también proporciona un poco de funciones de *Tablestakes* solo para asegurar que sea "viable" como producto.

El MVP también es una hipótesis. Podría ser lo suficientemente bueno para encontrar un mercado o no.

Se muestra el caso en el que cada cliente potencial con el que se interactúa solicita una funcionalidad distinta. Esto muestra que **aún no se encuentra un mercado** para el producto.



Si por el contrario, cada respuesta apunta a la misma funcionalidad, entonces tiene sentido revisar la **hipótesis cliente/problema/solución**.

Se ejecuta un pivot, un MVP2, centrado en la nueva hipótesis basada en el aprendizaje anterior

Supongamos que MVP2 es exitoso. Se desea aumentar el crecimiento y se busca una penetración más profunda de los primeros usuarios además de atraer nuevos clientes.

En función del **feedback recopilado y la investigación realizada**, se identifican un par de áreas que potencialmente pueden traer este crecimiento.



Algunos de ellas amplían su propuesta de valor única y otras hacen que su producto actual sea más robusto.

MMF → Minimum Marketable Feature



En el caso de áreas con una fuerte indicación de valor, se puede definir un MMF para encontrar la **pieza mínima que pueda empezar a traer crecimiento**.

Supone una alta certeza de que existe valor en esta área y que sabemos cuál debe ser el producto para proporcionar este valor.

La razón para dividir una característica grande en MMF más pequeños es principalmente el tiempo de comercialización (Time to market) y la capacidad de aportar valor en muchas áreas.



Una indicación de que está trabajando en MMF es que cuando al liberarse uno se siente cómodo trabajando en el próximo MMF en esa área.

MVF → Minimum Viable Feature

Se refiere a la característica mínima que aún puede ser viable **para uso real y aprendizaje de los usuarios reales**.

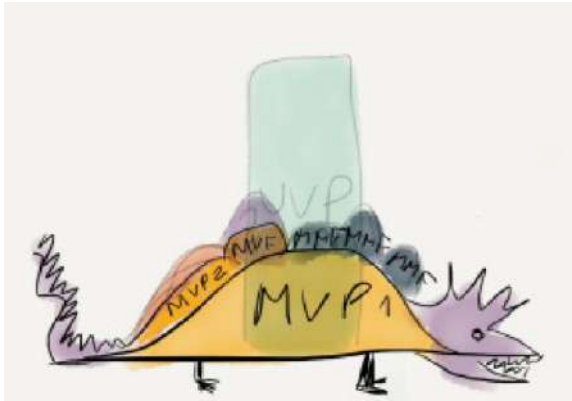
Si el MVF resulta exitoso (**hit gold**), puede desarrollar más MMF en esa área.

Si no es así, puede usar otro enfoque hacia esa área de características, o en algún momento buscar otra ruta de crecimiento

Esencialmente, el MVF es una versión mini del MVP



El modelo completo



El producto se cultiva en mercados inciertos al intentar varios MVP. Cuando se logra ajustar el producto en el mercado de productos se combinan MMF y MVF según el nivel de incertidumbre del negocio/requisitos en las áreas en las que se está enfocando.

Si bien los MVP/MMF/MVF son atómicos desde una perspectiva empresarial, pueden ser bastante grandes desde la perspectiva de la implementación.



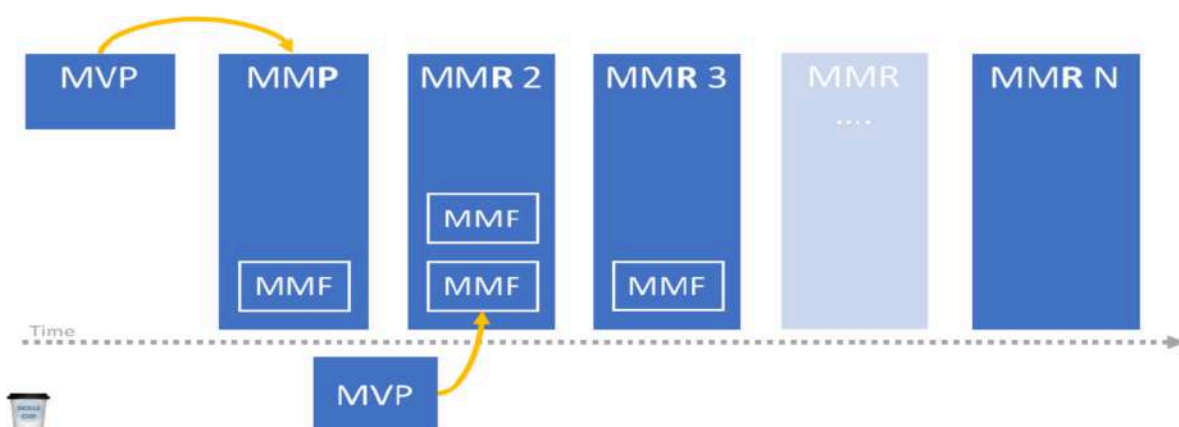
El dinosaurio se obtiene cortando cada una de esas piezas en pequeñas porciones destinadas a reducir el riesgo de ejecución/tecnología.

Productos mínimos para la gestión de productos

Estos son:

- **Minimum Viable Feature (MVF)**
 - Versión mini del MVP
 - Característica a pequeña escala que se puede construir e implementar rápidamente, utilizando recursos mínimos, para una población objetivo para probar la utilidad y adopción
- **Minimum Viable Product (MVP)**
 - Es la versión de un nuevo producto creado con el menor esfuerzo posible
 - Mas cercano a los prototipos que a una versión real del producto
 - Dirigido a un subconjunto de clientes potenciales, respondiendo a una hipótesis

- Utilizado para obtener aprendizaje validado por los clientes potenciales
- **Minimum Marketable Feature (MMF)**
 - Es la pieza más pequeña de funcionalidad que puede ser liberada
 - Tiene valor tanto para la organización como para los usuarios
 - Es parte de un MMR o de un MMP
- **Minimum Marketable Product (MMP)**
 - El primer release de un MMR, que está dirigido a primeros usuarios
 - Está focalizado en características clave que satisfarán a este grupo clave
- **Minimum Release Feature (MRF)**
 - Características mínimas debe tener un producto para salir al mercado. Es una versión del producto que se publica y que el cliente puede usar.
- **Minimum Marketable Release (MMR)**
 - Release de un producto que tiene el conjunto de características más pequeño posible.
 - El incremento más pequeño que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales.
 - **MMP = MMR1**



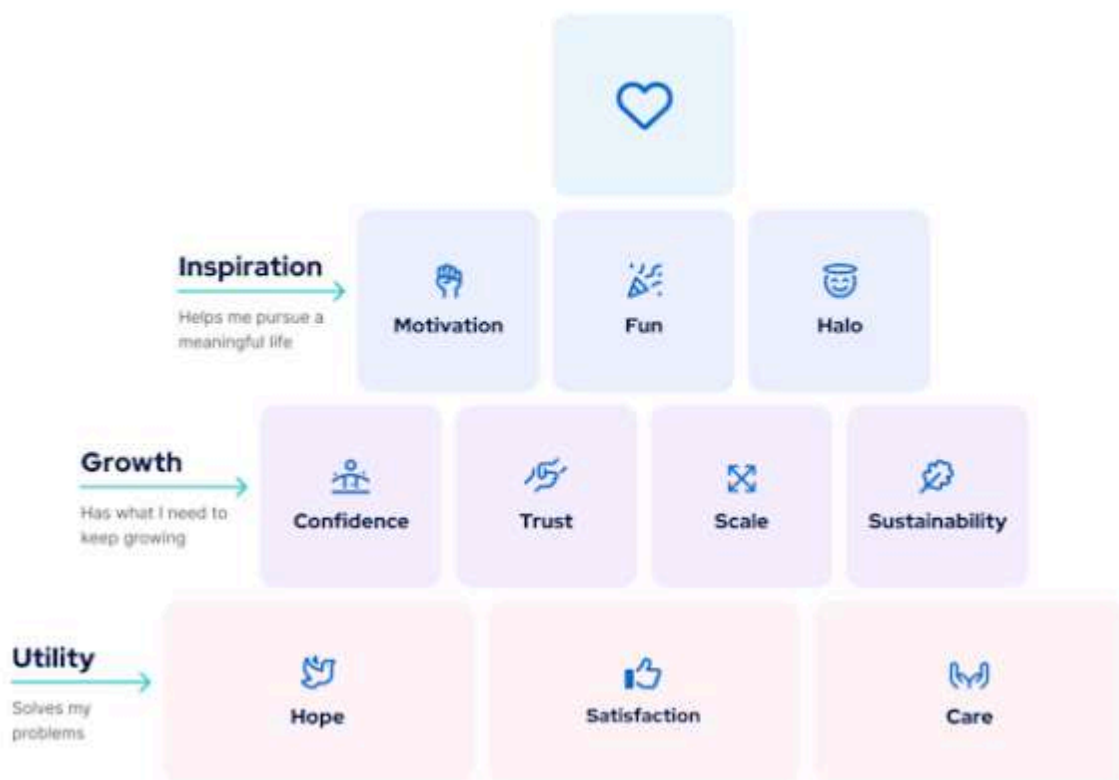
Minimal Lovable Product

Es la versión más simple de un producto que genera una conexión emocional en los usuarios, enfatizando la satisfacción del usuario y la diferenciación.



La idea es crear un producto viable mínimo que, además de proporcionar información al equipo de trabajo para el desarrollo futuro, apele a la emocionalidad del usuario para lograr lo más difícil: enamorarlo.

10 building blocks of lovability



Claves para definir un MLP

1. Entender y conocer al público objetivo
2. Definir una propuesta que aporte valor a las necesidades y expectativas del usuario
 - a. ¿En qué y cómo me diferencio?
 - b. ¿Qué valor añadimos al usuario?
 - c. ¿Cuál es el aspecto novedoso del producto?
3. Resolver al menos una de las necesidades del usuario de manera excelente

4. Sorprender al usuario, añadiendo un factor sorpresa en la parte visual o funcional del producto

¿Cómo crear un MLP?

- Sienta la curiosidad
 - No adivine, pregunte
- Conozca su mercado
 - Satisfaga las necesidades de sus clientes objetivo antes de expandirse a otros
- Defina su propósito
- Céntrese en el conjunto
 - Recuerde que su producto no es solo un conjunto de características, sino también todos los puntos de contacto con su empresa a lo largo del recorrido del cliente
- Construya para durar
- Dedique esfuerzo
- Mida el cariño

Aspecto	Funcionalidad	Usabilidad	Deleite
Rol	Resolver el problema del usuario	Asegurar facilidad de uso	Evocar emociones positivas
Foco clave	Aspectos core	Interface Intuitiva	Interacciones Intuitivas
Beneficio Principal	Satisfacer necesidades básicas	Reducir fricción	Fomentar la lealtad
Ejemplo	App de streaming de música	App con navegación simple	App con playlists personalizadas

MLP, MVP y MMP

MVP vs. MLP

Producto Mínimo Viable (MVP). Se centra en lograr su función principal para que el concepto del producto pueda validarse de la manera más económica y rápida posible. Este suele ser el enfoque predilecto de las empresas que tienen prisa por lanzar un producto al mercado.

Producto Mínimo Adorable (MLP). Se centra en características que no solo son funcionales o útiles, sino que también son lo suficientemente impactantes como para deleitar a los usuarios. Esto requiere una investigación de usuarios sobre lo que los clientes consideran valioso y, por lo tanto, su lanzamiento llevará más tiempo que un MVP.

MMP vs. MLP

Producto Mínimo Comercializable (MMP). Es un producto listo para su lanzamiento al mercado. Ya es capaz de cubrir una amplia gama de funcionalidades, atendiendo a grandes bases de usuarios, e incluso podría ser una versión posterior de un MVP anterior (aunque probablemente aún necesite ajustes de UX/UI).

Producto Mínimo Adorable (MLP). Es un producto que prioriza la interacción del usuario y enfatiza la creación de una conexión emocional con los clientes que lo prueban. Esto contrasta marcadamente con los MMP, que se centran en un mayor atractivo para el mercado y la disponibilidad para el lanzamiento.

50

05. Componentes de proyecto

🕒 Date Created @20 de septiembre de 2025 11:57

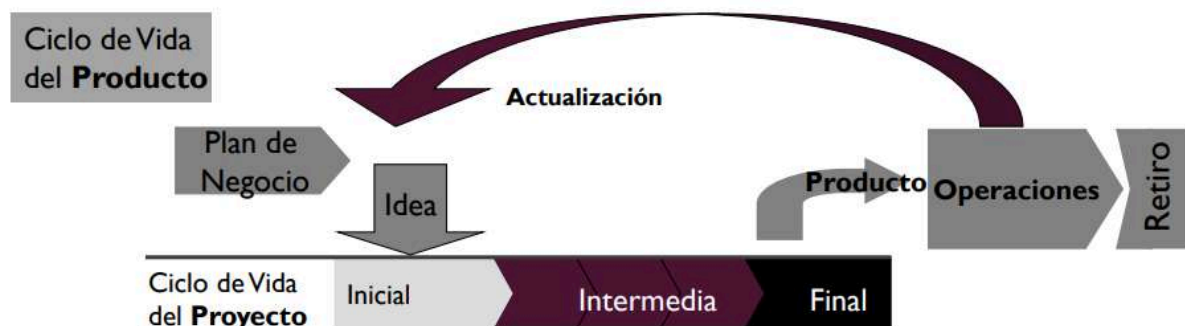


"El proceso es el marco metodológico, el proyecto es la unidad de trabajo temporal que lo implementa, y el producto es el resultado tangible que se entrega." dijo Chatgipipi

Ciclo de vida de proyectos y productos

El ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto, ya que el ciclo del proyecto equivale a la duración desarrollo del software. En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar

Es posible que un producto tenga varios proyectos en su ciclo de vida, debido a, constantes cambios y/o actualizaciones que se vayan realizando.



Proyecto de desarrollo de software

Es un esfuerzo temporal que requiere del acuerdo de un conjunto de especialidades y recursos para la creación de un producto, servicio o resultado único. Es una unidad organizativa para adaptar el marco teórico del proceso, dependiendo de las personas y recursos con los que se cuente.



Define el proceso y tareas que se van a realizar, el personal que se encargará de las actividades y los mecanismos que se implementarán para valorar riesgos, controlar el cambio y evaluar la calidad.

Características

Orientación a objetivos

- Los proyectos están dirigidos a obtener resultados y ello se refleja a través de objetivos.
- Los objetivos no deben ser ambiguos
- Los objetivos guían al proyecto
- Los objetivos deben ser claros y alcanzables

Duración limitada

Esto implica que un proyecto tiene un principio y un fin, liberando los recursos y personas.

Cuando se alcanzan los objetivos del proyecto, este llega a su fin.

Tareas interrelacionadas basadas en esfuerzos y recursos

Se definen las tareas, sus dependencias, los recursos y personas asignadas, permitiendo manejar la complejidad inherente de los sistemas.

Únicos

El resultado de un proyecto será único e irrepetible para cualquier otro proyecto, por más parecido o iguales que sean sus elementos componentes

Administración de proyectos

Es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer los requerimientos del proyecto.

Esta disciplina tiene dos grandes actividades:

- Planificación.
- Monitoreo y control.

Incluye:

- Identificar los requerimientos
- Establecer objetivos claros y alcanzables
- Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los stakeholders

La triple restricción - Enfoque tradicional

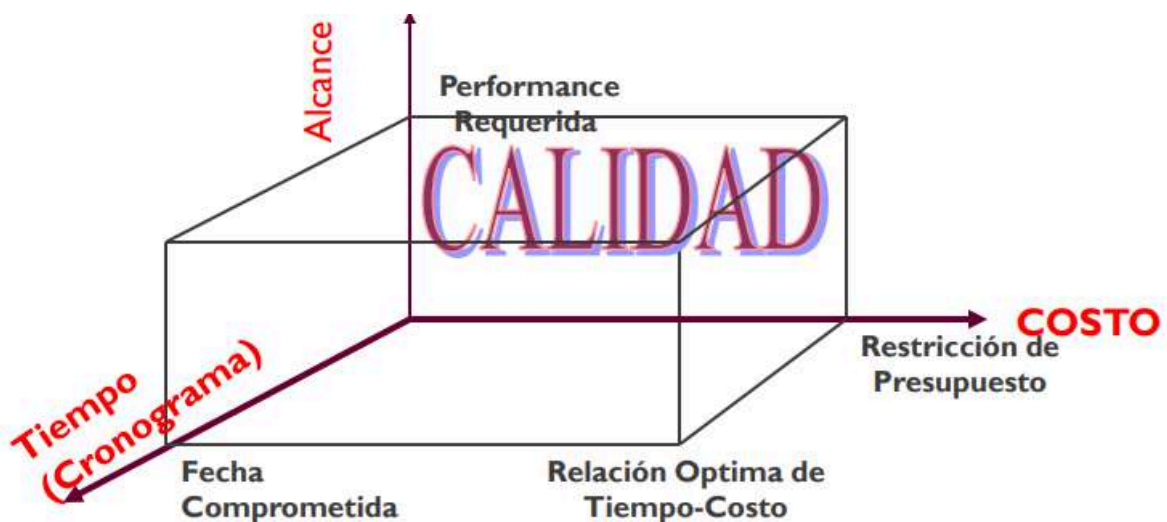
Las variables de restricción de un proyecto son:

- Alcance → ¿Qué está el proyecto tratando de alcanzar?
- Tiempo → ¿Cuánto tiempo debería llevar completarlo?
- Costos → ¿Cuánto debería costar?



El balance de estos tres factores afecta directamente la calidad del proyecto, y eso es responsabilidad del líder del proyecto

En la realidad, el costo y el tiempo del proyecto varían de forma directa con la definición del alcance del producto, es decir, el alcance es directamente proporcional al tiempo y al costo. Cuando el alcance aumenta (mayor cantidad de funcionalidades), el tiempo y dinero (costos) necesarios también deben aumentar, para lograr abordar un proyecto más grande.

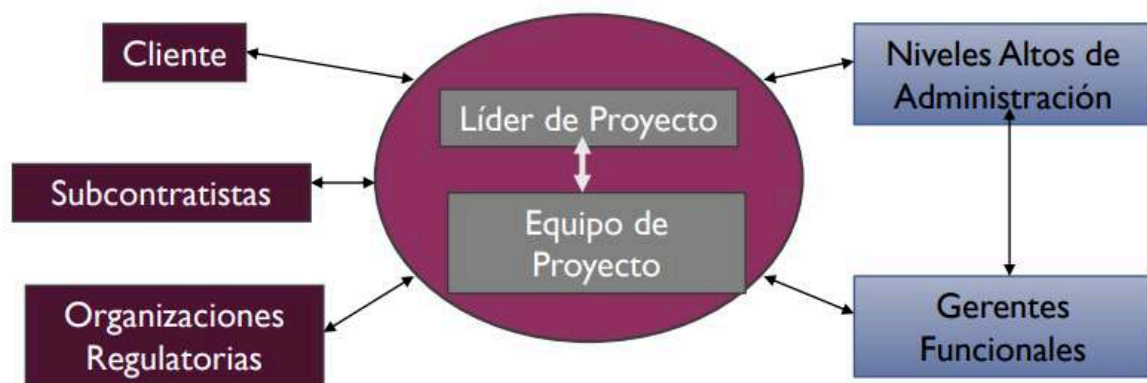


El problema en la aplicación de esta visión en enfoques tradicionales es que, al utilizar procesos definidos con ciclos de vida en cascada, los requerimientos se definen al comienzo del proyecto, por lo que cualquier modificación o cambio en el alcance del producto final, modifica el tiempo y presupuesto de este.

Líder del proyecto

Entre las responsabilidades del líder de proyecto se encuentran, en las nuevas metodologías:

1. Definir el alcance del proyecto.
2. Identificar a los involucrados, recursos y presupuesto.
3. Detallar las tareas, estimar los tiempos y requerimientos.
4. Identificar y evaluar riesgos.
5. Preparar planes de contingencia.
6. Controlar hitos y participar en las revisiones de las fases del proyecto.
7. Administrar el proceso de control de cambios.
8. Producir reportes de estado.



Equipo de proyecto

Es un grupo de personas comprometidas con alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables.

Entre sus características se encuentran:

- Cuentan con conocimientos, habilidades técnicas, motivación, experiencias y diversas personalidades.
- Usualmente son un grupo pequeño, esto permite lograr una comunicación efectiva entre los miembros del equipo.
- Trabajan en forma conjunta con espíritu de grupo, convirtiéndose en un equipo cohesivo donde prima la sinergia de este. Para eso, se debe lograr un equipo maduro y estable, es decir, que no se agrega o cambia un integrante constantemente.
- Sentido de responsabilidad como una unidad y se focalizan en el cumplimiento de las metas grupales.

Plan de proyecto - Roadmap

El plan de proyecto documenta:

- ¿Qué es lo que hacemos?
- ¿Cuándo lo hacemos?
- ¿Cómo lo hacemos?
- ¿Quién lo va a hacer?

Esta planificación implica:

- **Definición del alcance** (QUÉ)

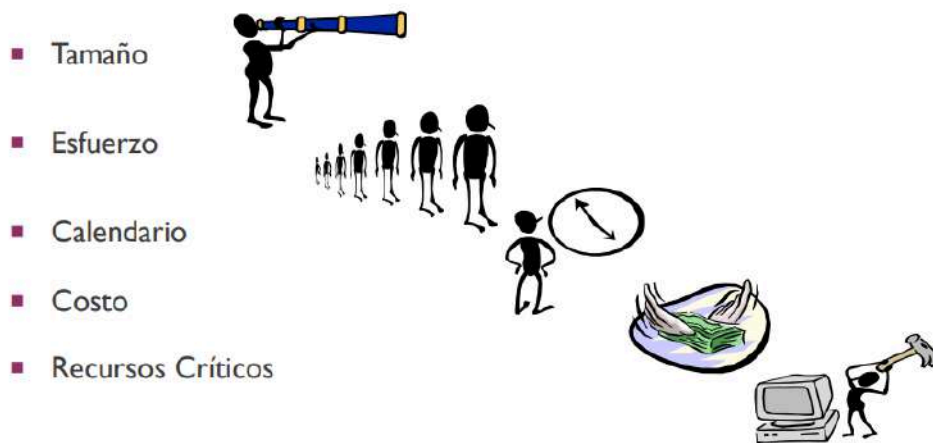
Alcance del Producto:

- Son todas las características que pueden incluirse en un producto o servicio.

Alcance del Proyecto:

- Es **todo el trabajo** y **solo el trabajo** que debe hacerse para entregar el producto o servicio con todas las características y funciones especificadas.

- **Definición del proceso y ciclo de vida** (CÓMO)
- **Estimación**



- **Gestión de riesgos**



- **Programación de proyectos**
- **Definición de controles**
- **Definición de métricas**

Este dominio se divide en métricas del proceso, del proyecto y del producto

Las métricas del proyecto se consolidan para crear métricas de proceso que sean públicas para toda la org. de software.

Métricas básicas para un proyecto de software



Tamaño del producto



Esfuerzo



Tiempo (Calendario)

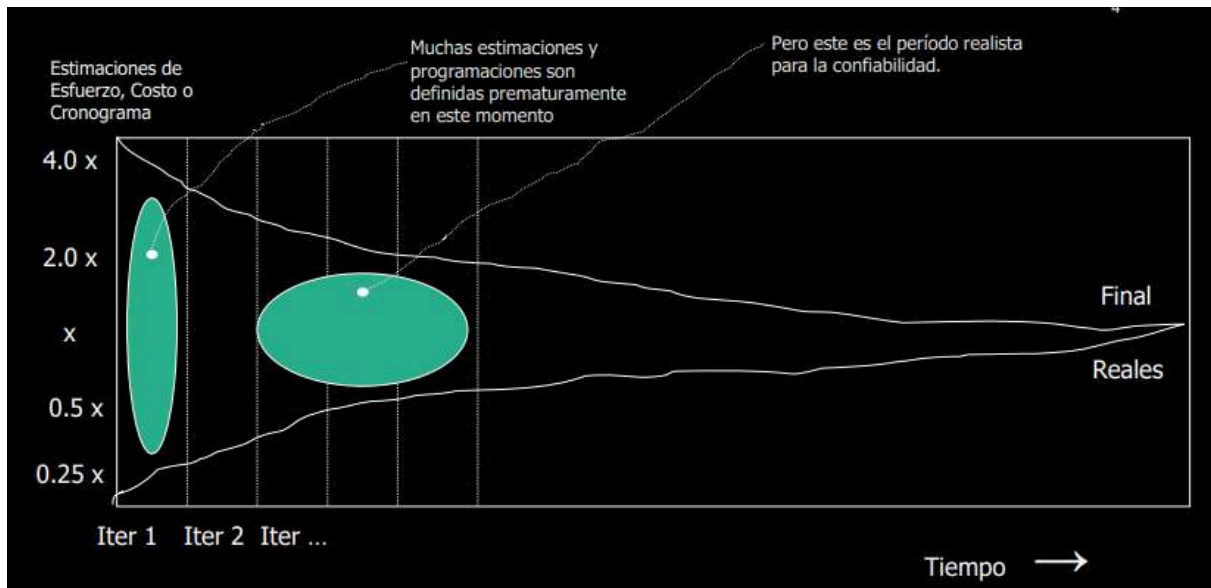


Defectos

06. Estimación del esfuerzo

🕒 Date Created @20 de septiembre de 2025 11:58

Objetivo de la estimación



Métodos

- Basados en la experiencia.
- Basados exclusivamente en los recursos.
- Método basado exclusivamente en el mercado.
- Basados en los componentes del producto o en el proceso de desarrollo.
- Métodos algorítmicos

Basados en la experiencia

- Datos históricos
- Juicio experto
 - Puro
 - Delphi

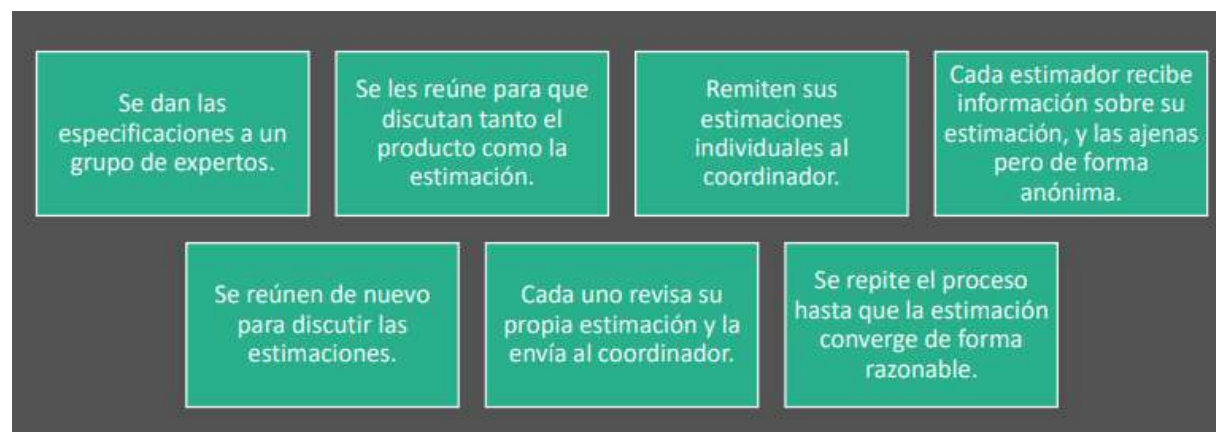
- Analogía

Juicio experto puro

Un experto estudia las especificaciones y hace su estimación, basándose fundamentalmente en sus conocimientos.

Juicio experto Delphi

Un grupo de personas son informadas y tratan de adivinar lo que costará el desarrollo tanto en esfuerzo, como en duración.

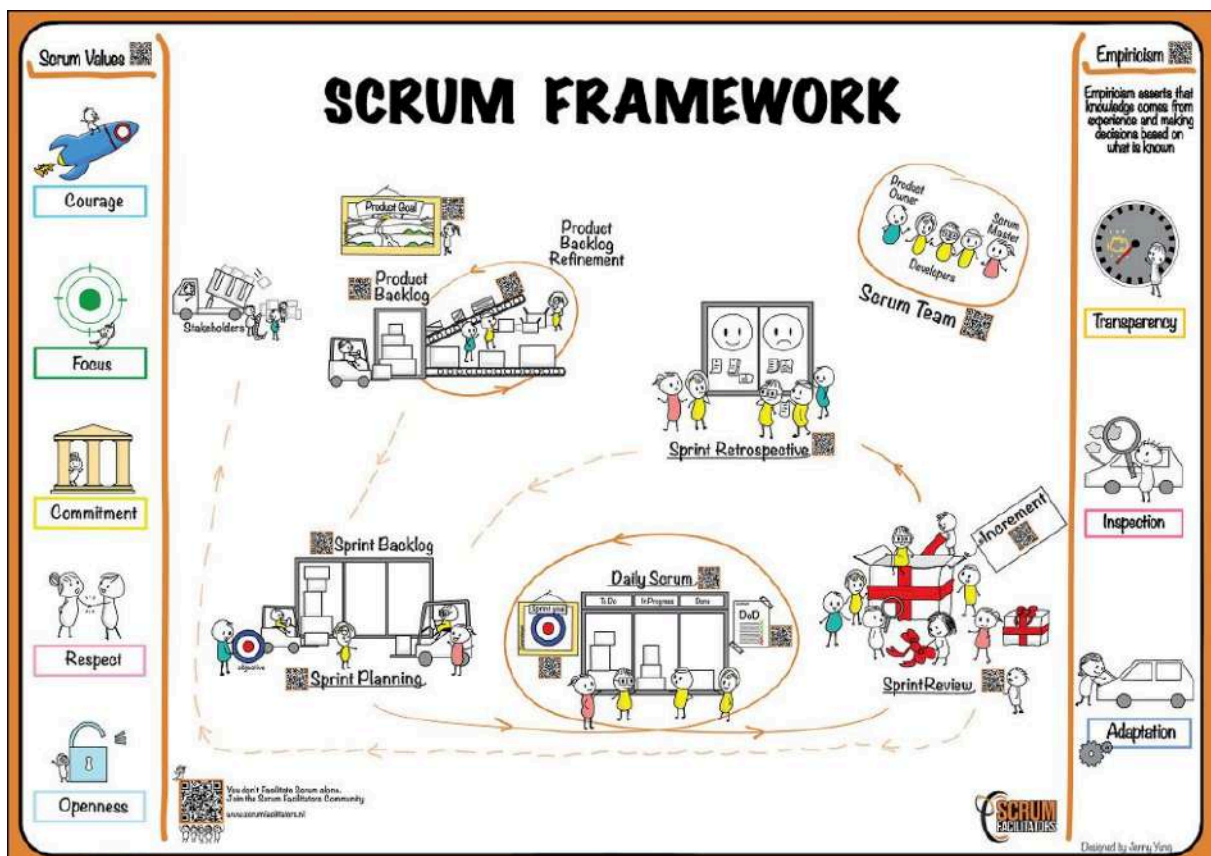


07. SCRUM

🕒 Date Created @20 de septiembre de 2025 11:58

Scrum

Es un marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos



Los **pilares** del empirismo (y por lo tanto de Scrum) son:

- Inspección

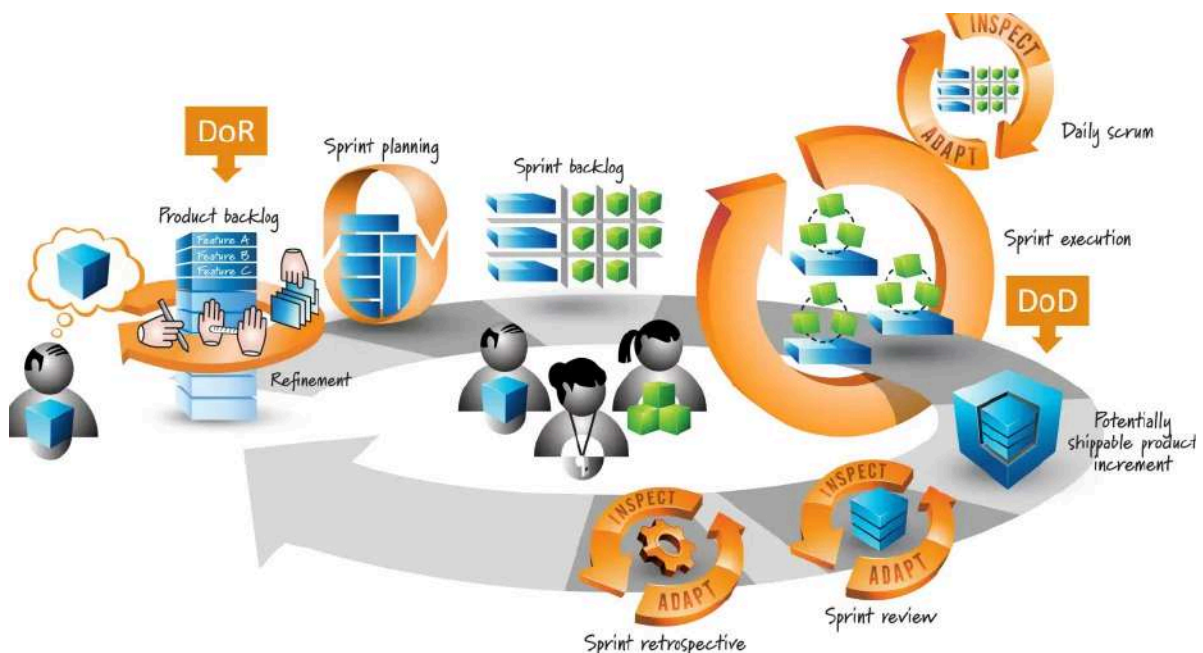
Se realiza por todo el equipo y permite detectar varianzas o problemas potencialmente indeseables, se debe inspeccionar el progreso hacia los objetivos acordados

- Transparencia

Permite el crecimiento como equipo y que el conocimiento que cada miembro del equipo tiene implícito que pase a formar parte del conocimiento de todo el equipo

- Adaptación

Se trata de la mejora continua, es la capacidad de adaptarse en función de los resultados de la inspección. Nos permite hacer ajustes en aspectos del proceso que se desvían fuera de los límites o si el producto es inaceptable.



Roles

Product Owner

Es responsable de maximizar el valor del producto resultante del trabajo del equipo de

Scrum y de la gestión eficaz de la pila del producto:

- Desarrollar y comunicar explícitamente el objetivo del producto
- Creación y comunicación clara de elementos de trabajo pendiente del producto
- Pedido de artículos de trabajo pendiente del producto

- Asegurarse de que el trabajo pendiente del producto sea transparente, visible y comprendido

Scrum Master

El Scrum Master es responsable de establecer Scrum tal como se define en la Guía de Scrum. Lo consigue ayudando a todos a comprender la teoría y la práctica de Scrum, tanto dentro del Equipo como en toda la organización.

Artefactos

Representan trabajo o valor. Están diseñados para maximizar la transparencia de la información clave.

Cada artefacto contiene un compromiso para garantizar que proporciona información que mejora la transparencia



El enfoque con el que se puede medir el progreso es:

- Trabajo pendiente del producto → Objetivo del producto.
- Sprint Backlog → Sprint Goal.
- Incremento de producto → Definición de Hecho

Product Backlog

El trabajo pendiente del producto es una lista emergente y ordenada de lo que se necesita para mejorar el producto. Es la única fuente de trabajo emprendida por el equipo Scrum.

Compromiso: Product Goal

Describe un estado futuro del producto que puede servir como objetivo para el equipo Scrum contra el cual planificar

Sprint Backlog

Se compone del objetivo sprint (por qué), el conjunto de elementos de trabajo pendiente de producto seleccionados para el Sprint (qué), así como un plan accionable para entregar el incremento (cómo)

Compromiso: Product Goal

Es el único objetivo para el Sprint. Aunque este sea un compromiso de los desarrolladores, proporciona flexibilidad en términos del trabajo exacto necesario para lograrlo

Incremento

Compromiso: DoD

Timebox



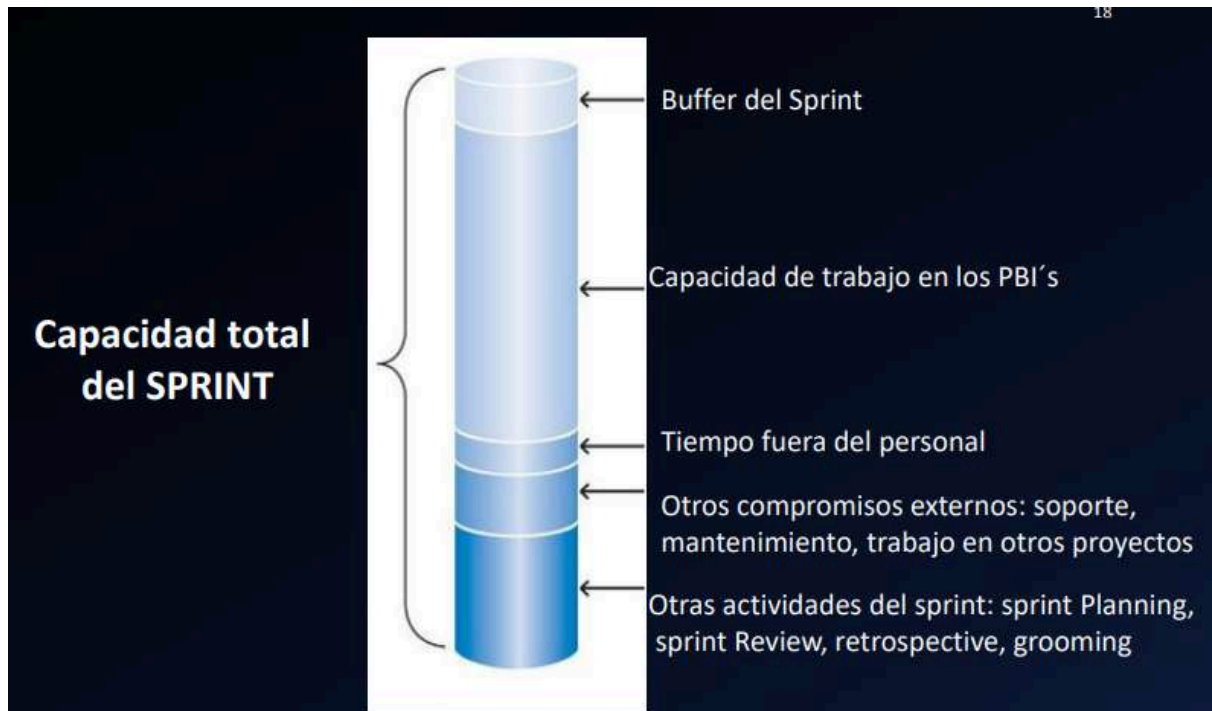
DoR → Definition of Ready

Definición de "Listo" (Ready)	
☐	Valor de negocio claramente expresada.
☐	Detalles suficientemente comprendidos por el Equipo de forma tal que puedan tomar una decisión informada sobre si pueden completar el ítem del product Backlog (PBI) .
☐	Dependencias identificadas y no hay dependencias externas que puedan impedir que el PBI se complete.
☐	El equipo ha sido asignado adecuadamente para completar el PBI.
☐	El PBI ha sido debidamente estimado y es lo suficientemente pequeño para ser completado en un Sprint.
☐	Los criterios de aceptación son claros y testeables.
☐	Los criterios de performance si hay, son claros y testeables.
☐	El equipo comprende como mostrar el PBI en la Sprint Review.

DoD → Definition of Done

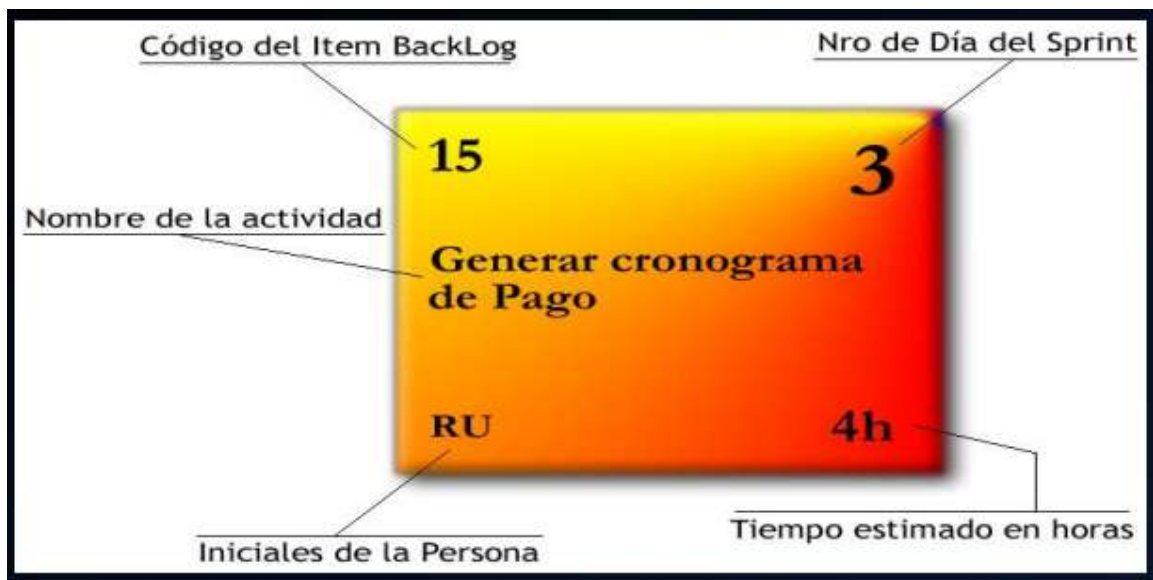
Definición de Hecho (DONE)	
☐	Diseño revisado
☐	Código Completo
☐	Código refactorizado
☐	Código con formato estándar
☐	Código Comentado
☐	Código en el repositorio
☐	Código Inspeccionado
☐	Documentación de Usuario actualizada
☐	Probado
☐	Prueba de unidad hecha
☐	Prueba de integración hecha
☐	Prueba de sistema hecha
☐	Cero defectos conocidos
☐	Prueba de Aceptación realizada
☐	En los servidores de producción

Capacidad del equipo



Herramientas

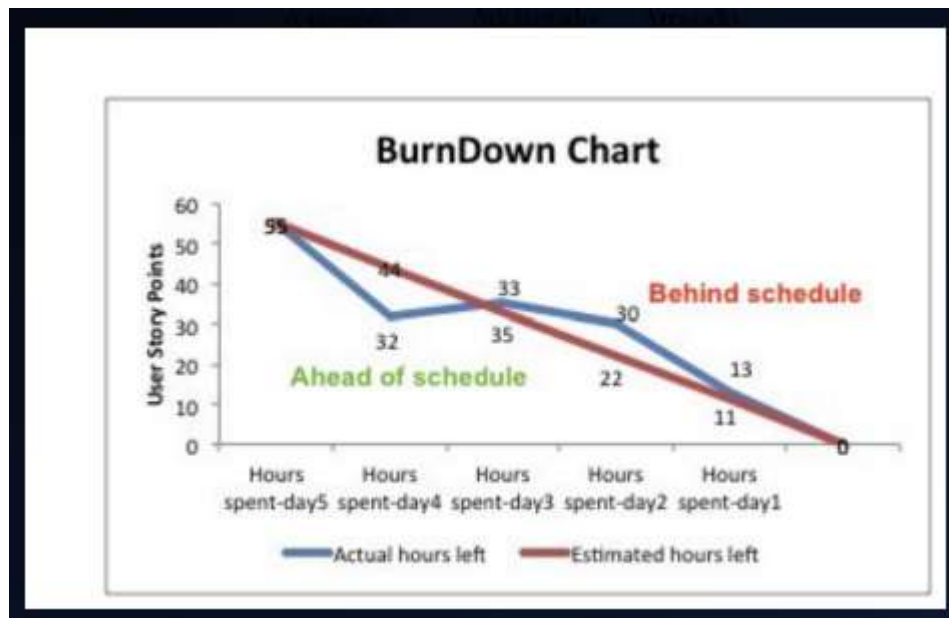
- Task card



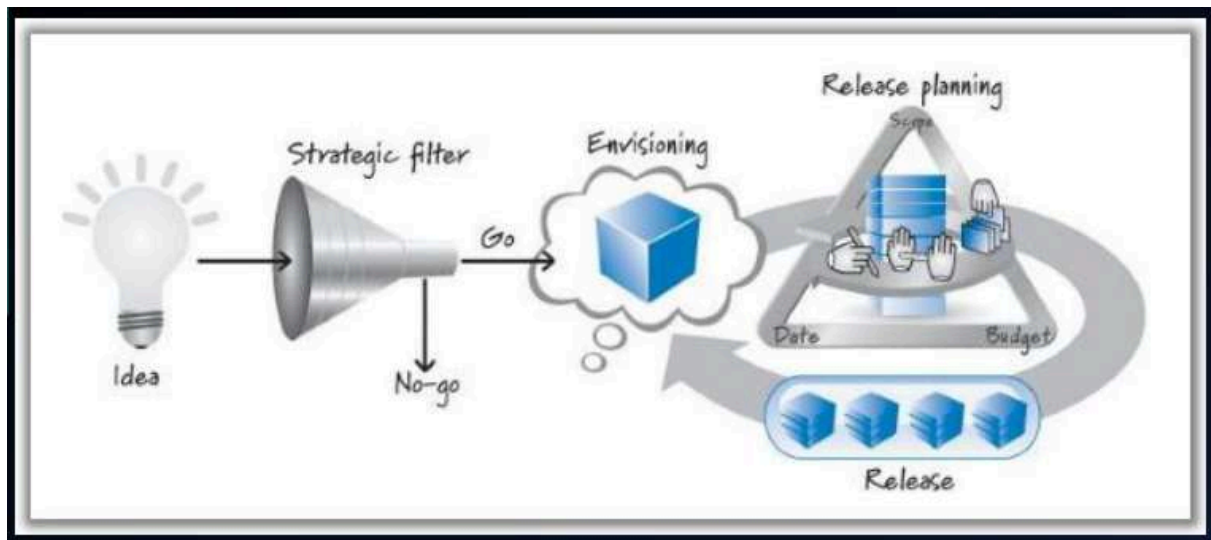
- Task board

Story	To Do		In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Test the... 8	Code the... DC 4	Test the... SC 6	Code the... D 8
	Code the... 2	Code the... 8	Test the... SC 8		Test the... SC 8
	Test the... 8	Test the... 4			Test the... SC 6
As a user, I... 5 points	Code the... 8	Test the... 8	Code the... DC 8		Test the... SC 8
	Code the... 4	Code the... 6			Test the... SC 6

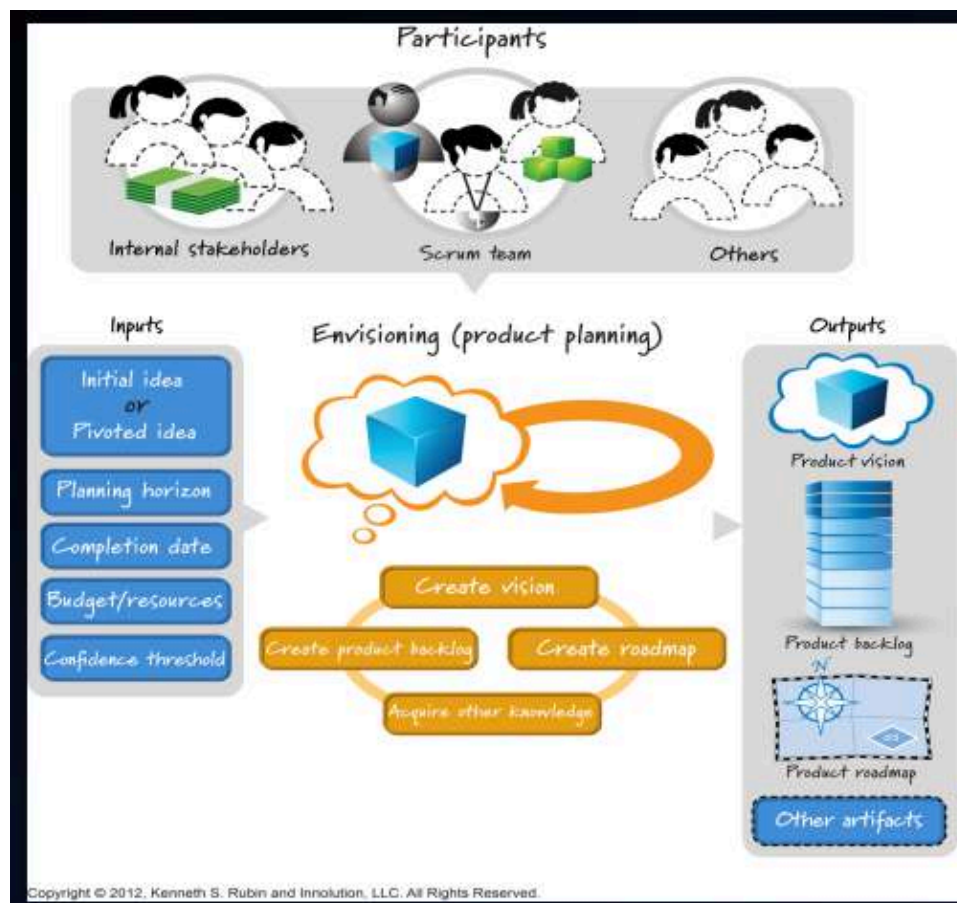
- Sprint burndown charts



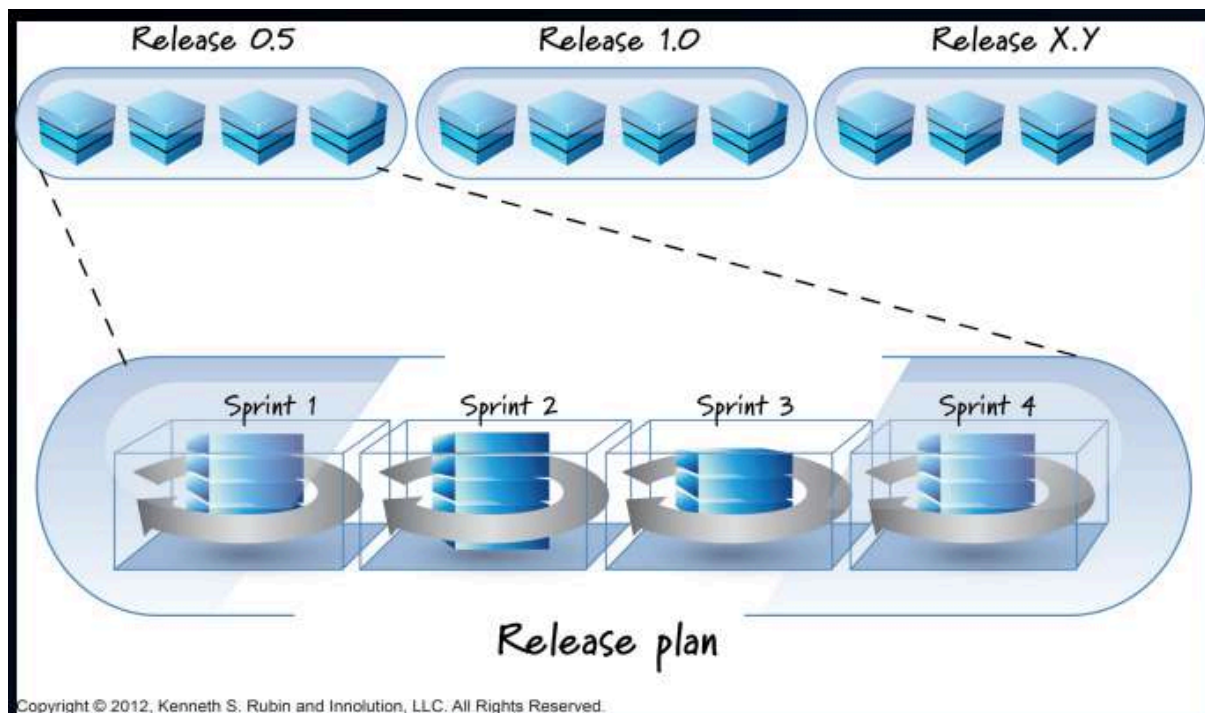
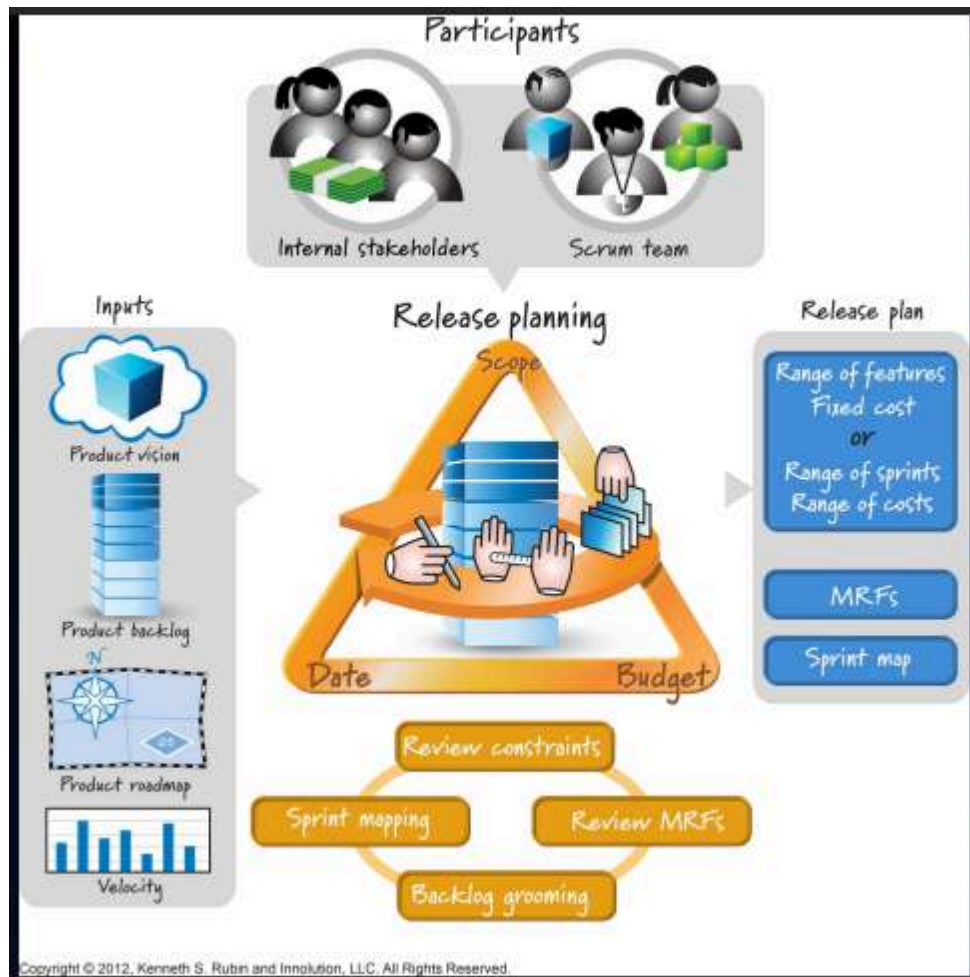
Planificación



De producto

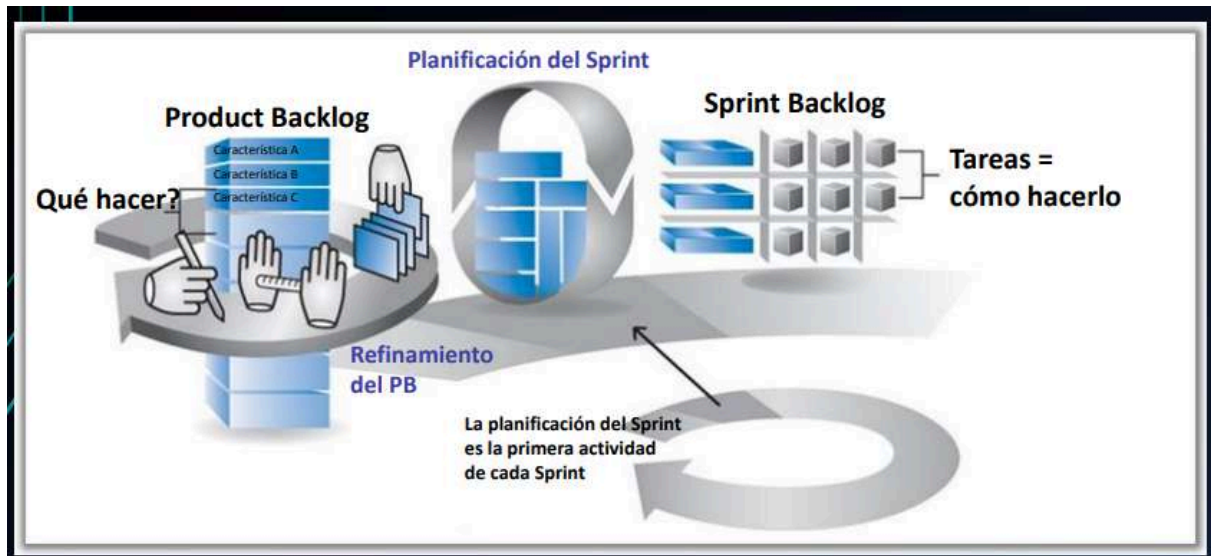


De release

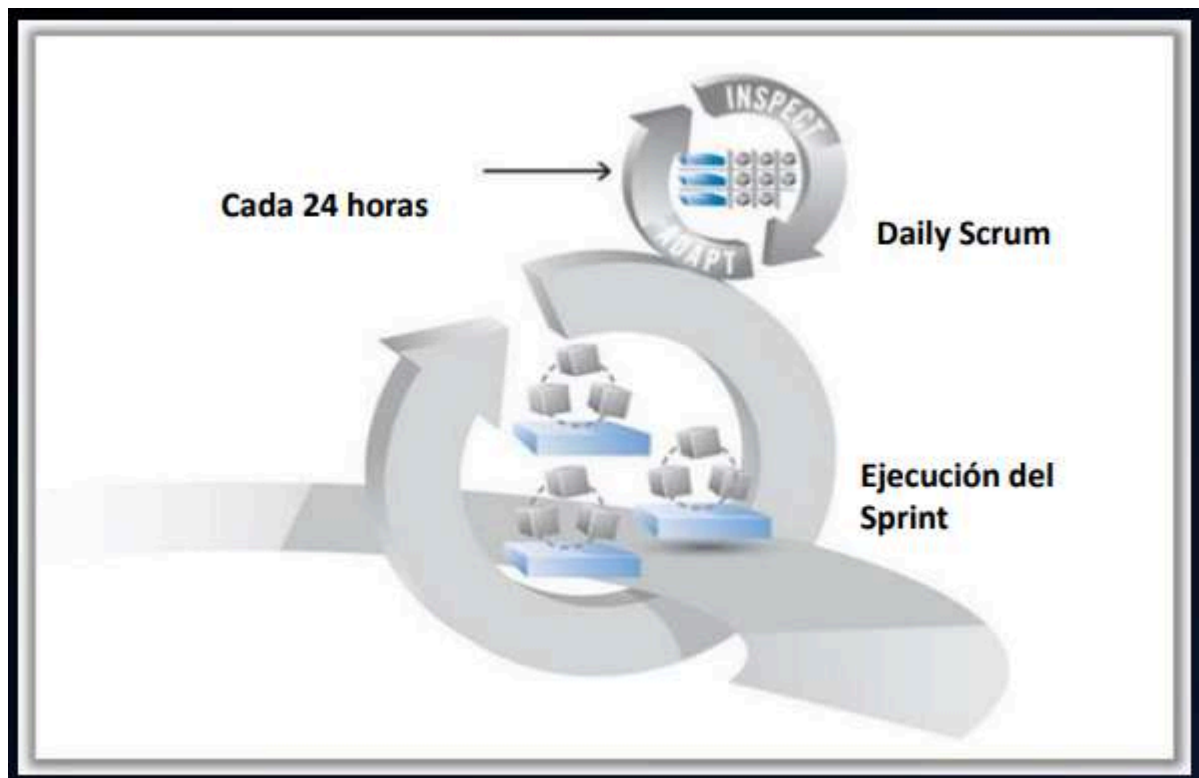


Releases y sprints

De iteración → Sprint Planning

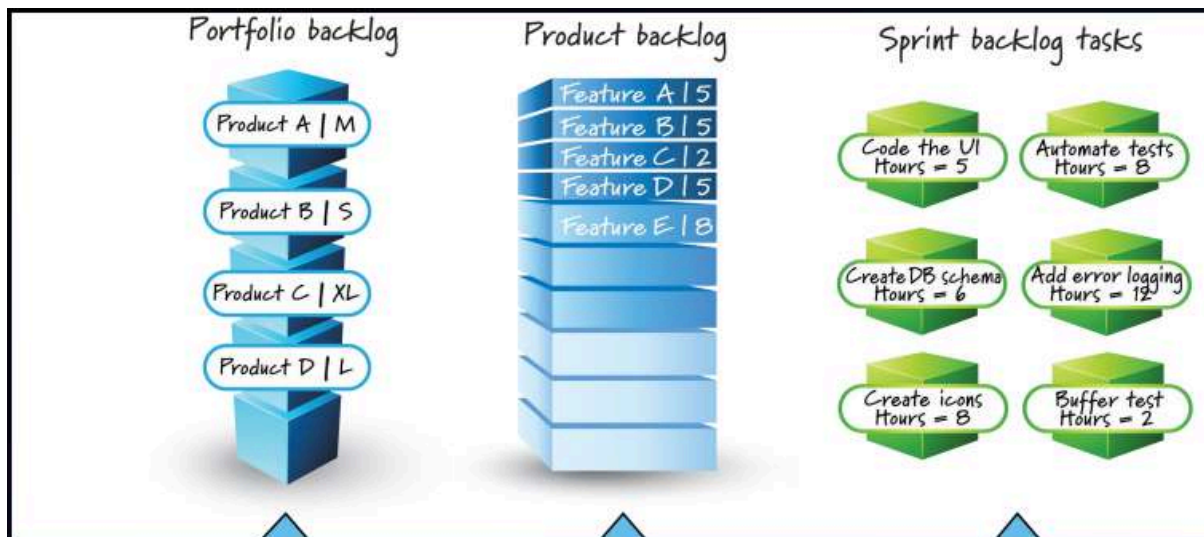


Diaria → Daily Scrum



Niveles de planificación

Nivel	Horizonte	Quién	Foco	Entregable
Portfolio	1 año o más	Stakeholders y Product Owners	Administración de un Portfolio de Producto	Backlog de Portfolio
Producto	Arriba de varios meses o más	Product Owner y Stakeholders	Visión y evolución del producto a través del tiempo	Visión de Producto, Roadmap y características de alto nivel
Release	3 (o menos) a 9 meses	Equipo Scrum, Stakeholders	Balancear continuamente el valor de cliente y la calidad global con las restricciones de alcance, cronograma y presupuesto	Plan de Release
Iteración	Cada iteración (de 1 semana a 1 mes)	Equipo Scrum	Que aspectos entregar en el siguiente sprint	Objetivo del Sprint y Sprint Backlog
Día	Diaria	Equipo Scrum (al menos los que trabajan en IPB)	Cómo completar lo comprometido	Inspección del progreso actual y adaptación a la mejor forma de organizar el siguiente día de trabajo



Métricas

- Running Tested Features
- Capacidad

Horas de trabajo disponibles por día * Días disponibles de Iteración =
Capacidad del equipo

- Velocidad del equipo